Capstone Project
Cricket match prediction

By: POONAM TEKWANI

Date: 14th July 2024

PGPDSBA.0.JULY23.B

Table of contents:

Introduction - What did you wish to achieve while doing the project ?	5 - 6
EDA - Uni-variate / Bi-variate / Multi-variate analysis to understand relationship b/w variables. - Both visual and non-visual understanding of the data.	8 - 27
Data Cleaning and Pre-processing - Approach used for identifying and treating missing values and outlier treatment (and why) - Need for variable transformation (if any) - Variables removed or added and why (if any)	**27 - 30**
Model building - Clear on why was a particular model(s) chosen. - Effort to improve model performance.	36 - 49
Model validation - How was the model validated ? Just accuracy, or anything else too ?	48 - 49
Final interpretation / recommendation - Very clear and crisp on what recommendations do you want to give to the management / client.	51 - 52

List of figures:

Data summarization FIG 1

Basic information about dataset, its datatypes: FIG2

Checking datatypes of each variable: Fig 3

Checking null values in dataset: FIG 4

After treating Null values and dropping some of them: Fig 5

Statistical Information about dataset: FIG 6

FIG 7 countplot of result

FIG 8 Histogram of Average Team Age

FIG 9 Count plot of match format

FIG 10 Histogram & boxplot of Bowlers_in_team

FIG 11 Histogram & boxplot of All_rounder_in_team

FIG 12 Countplot of opponents

FIG 13 countplot of season

FIG 14 countplot of match light type

FIG 15 countplot of first selection

FIG 16 Histogram & boxplot of extra_bowls_opponent

FIG 17 countplot of playes scored zero

FIG 18 countplot of player highest wicket

FIG 19 Boxplot of Max_wicket_taken_1over by Match_format (hue by Result)

FIG 20 Boxplot of player_highest_run by Match_format (hue by Result)

Fig 21 Boxplot of Audience_number by Match_light_type (hue by Match_format)

FIG 22: Boxplot of Bowlers_in_team by Match_format (hue by Result)

Correlation: Fig 23

Correlation after encoding: FIG 24

Checking if there are any outliers present in dataset. FIG 25

After treating outliers visual representation: FIG 26

FIG 27 class distribution of results

Fig 28 resampled class distribution

FIG 29 Accuracy metrics Random forest classifier

FIG 30 class distribution after smote

FIG 31 Clustering metrics

Fig 32 logistic regression Model Evaluation Metrics

Fig 33 decision tree Model Evaluation Metrics

Fig 34 Random Forest Model Evaluation Metrics

Fig 35 Visual Representation Heatmap

Fig 36 Feature importances

Fig 37 visual representation model evaluation metrices

Fig 38 visual representation of feature importances

Fig 39 accuracy metrics for random forest classifier

Fig 40 accuracy metrics for gradient boosting

Fig 41 feature importance for random forest

1) Introduction of the business problem

a) Defining problem statement

The Board of Control for Cricket in India (BCCI) aims to leverage data analytics to enhance the Indian cricket team's performance. The primary objectives are to:

1. **Develop Predictive Models**: Create machine learning models to accurately predict match outcomes (win/loss) based on historical match data.
2. **Provide Strategic Recommendations**: Extract actionable insights and suggest unique, match-specific strategies to improve the chances of winning.
3. **Optimize Performance for Upcoming Matches**: Predict outcomes for the next 10 matches and propose adjustments for predicted losses to ensure wins, considering match-specific conditions and opponents.

This project will involve data cleaning, model training, prediction, strategy formulation, and reporting to support the Indian cricket team in achieving consistent victories.

b) Need of the study/project

Enhance Indian cricket team's performance through data-driven insights, adaptive strategies, and evidence-based decision-making. This project aims to provide a competitive edge, prepare for upcoming matches, and inform stakeholders with actionable insights.

c) Understanding business/social opportunity

1. **Performance Enhancement**:
    - Improve the Indian cricket team's performance through data-driven insights and tailored strategies, leading to more victories on the field.

2. **Strategic Advantage**:
    - Gain a competitive edge by using predictive models and unique match strategies, ensuring adaptability and success against different opponents and match conditions.
3. **Fan Engagement**:
    - Increase fan interest and support through enhanced team performance, driving social and commercial opportunities for cricket.
4. **Investment Appeal**:
    - Attract investments from sponsors and stakeholders drawn to a winning team, leading to lucrative sponsorship deals and partnerships.
5. **Inspiration for Future Generations**:
    - Inspire future athletes by showcasing the impact of data-driven decision-making in sports, fostering a culture of innovation and excellence.

In essence, this project offers both business and social opportunities by improving team performance, engaging fans, attracting investments, and inspiring the next generation of athletes.

2)Data Report

a) Understanding how data was collected in terms of time, frequency and methodology

Data Collection Overview

Time Frame:

- Data collection occurred over several years, encompassing historical match data from various cricket tournaments and series involving the Indian cricket team.

- The time frame includes matches played across different seasons, providing a longitudinal view of team performance.

Frequency:

- Match data was collected for each match played by the Indian cricket team, ensuring a comprehensive dataset.
- The frequency of data collection corresponds to the scheduling of cricket matches, including test matches, T20 matches, and one-day internationals (ODIs).

Methodology:

- Data collection methods included manual data entry and automated retrieval processes.
- Manual data entry involved recording match details such as venue, date, format, opponent, and outcome from reputable sources like cricketing websites and official match reports.
- Automated retrieval utilized web scraping techniques to extract match statistics from online databases and APIs provided by cricket governing bodies.
- Rigorous quality control measures were implemented to validate the accuracy and consistency of collected data, including cross-referencing against multiple sources and error detection algorithms.

In summary, the data collection process was comprehensive and systematic, spanning multiple years and covering various match formats to provide a robust dataset for analysis and modelling.

b) Visual inspection of data (rows, columns, descriptive details)

Data summarization FIG 1

	Game_number	Result	Avg_team_Age	Match_light_type	Match_format	Bowlers_in_team	Wicket_keeper_in_team	All_rounder_in_team	First_selection	Oppor
0	Game_1	Loss	18.0	Day	ODI	3.0	1	3.0	Bowling	Srila
1	Game_2	Win	24.0	Day	T20	3.0	1	4.0	Batting	Zimba
2	Game_3	Loss	24.0	Day and Night	T20	3.0	1	2.0	Bowling	Zimba
3	Game_4	Win	24.0	NaN	ODI	2.0	1	2.0	Bowling	Ke
4	Game_5	Loss	24.0	Night	ODI	1.0	1	3.0	Bowling	Srila
5	Game_6	Win	24.0	Night	ODI	3.0	1	4.0	Batting	Srila
6	Game_7	Loss	24.0	Day and Night	T20	2.0	1	2.0	Batting	Zimba
7	Game_8	Win	24.0	Night	ODI	3.0	1	2.0	Batting	Ke
8	Game_9	Loss	24.0	Day	ODI	3.0	1	3.0	Bowling	Srila
9	Game_10	Win	24.0	Day	T20	3.0	1	4.0	Batting	Zimba

Shape of dataset

```
(2930, 23)
```

Our dataset consist of 2930 rows and 30 columns.

Basic information about dataset, its datatypes: FIG2

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2930 entries, 0 to 2929
Data columns (total 23 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Game_number             2930 non-null   object
 1   Result                  2930 non-null   object
 2   Avg_team_Age            2833 non-null   float64
 3   Match_light_type        2878 non-null   object
 4   Match_format            2860 non-null   object
 5   Bowlers_in_team         2848 non-null   float64
 6   Wicket_keeper_in_team   2930 non-null   int64
 7   All_rounder_in_team     2890 non-null   float64
 8   First_selection         2871 non-null   object
 9   Opponent                2894 non-null   object
 10  Season                  2868 non-null   object
 11  Audience_number         2849 non-null   float64
 12  Offshore                2866 non-null   object
 13  Max_run_scored_1over    2902 non-null   float64
 14  Max_wicket_taken_1over  2930 non-null   int64
 15  Extra_bowls_bowled      2901 non-null   float64
 16  Min_run_given_1over     2930 non-null   int64
 17  Min_run_scored_1over    2903 non-null   float64
 18  Max_run_given_1over     2896 non-null   float64
 19  extra_bowls_opponent    2930 non-null   int64
 20  player_highest_run      2902 non-null   float64
 21  Players_scored_zero     2930 non-null   object
 22  player_highest_wicket   2930 non-null   object
dtypes: float64(9), int64(4), object(10)
memory usage: 526.6+ KB
```

Checking duplicates in dataset:

```
Number of duplicate rows: 0
Duplicate rows:
Empty DataFrame
Columns: [Game_number, Result, Avg_team_Age, Match_light_type, Match_format, Bowlers_in_team, Wicket_keeper_in_team, All_rounde
r_in_team, First_selection, Opponent, Season, Audience_number, Offshore, Max_run_scored_1over, Max_wicket_taken_1over, Extra_bo
wls_bowled, Min_run_given_1over, Min_run_scored_1over, Max_run_given_1over, extra_bowls_opponent, player_highest_run, Players_s
cored_zero, player_highest_wicket]
Index: []

[0 rows x 23 columns]
```

No duplicate entries present in the dataset.

Checking datatypes of each variable: Fig 3

```
Data types of each column:
Result                      int8
Avg_team_Age              float64
Match_light_type            int8
Match_format                int8
Bowlers_in_team           float64
All_rounder_in_team       float64
First_selection             int8
Opponent                    int8
Season                      int8
Max_run_scored_1over      float64
Max_wicket_taken_1over      int64
Extra_bowls_bowled        float64
Min_run_given_1over         int64
Min_run_scored_1over      float64
Max_run_given_1over       float64
extra_bowls_opponent        int64
player_highest_run        float64
Players_scored_zero         int64
player_highest_wicket       int8
dtype: object
```

Checking null values in dataset: FIG 4

```
Game_number                 0
Result                      0
Avg_team_Age               97
Match_light_type           52
Match_format               70
Bowlers_in_team            82
Wicket_keeper_in_team       0
All_rounder_in_team        40
First_selection            59
Opponent                   36
Season                     62
Audience_number            81
Offshore                   64
Max_run_scored_1over       28
Max_wicket_taken_1over      0
Extra_bowls_bowled         29
Min_run_given_1over         0
Min_run_scored_1over       27
Max_run_given_1over        34
extra_bowls_opponent        0
player_highest_run         28
Players_scored_zero         0
player_highest_wicket       0
dtype: int64
```

Yes we have null values present in 15 columns, we will treat them by imputing or maybe dropping nan values.

Missing Value Treatment

There are Null Values in the following variables:

a) 'Avg_team_Age',
b) ‘match_light_type’,
c) ‘match_format’,
d) 'Bowlers_in_team',
e) 'All_rounder_in_team',
f) 'Audience_number',
g) 'Max_run_scored_1over',
h) 'Extra_bowls_bowled',
i) 'Min_run_scored_1over',
j) 'Max_run_given_1over',

k) 'player_highest_run'

The missing values are treated with mean , median and mode of the respective column.

Null value imputation is a crucial step in data preprocessing to ensure the accuracy and reliability of subsequent analysis and modeling.

After treating Null values and dropping some of them: Fig 5

```
Game_number                0
Result                     0
Avg_team_Age               0
Match_light_type           0
Match_format               0
Bowlers_in_team            0
Wicket_keeper_in_team      0
All_rounder_in_team        0
First_selection            0
Opponent                   0
Season                     0
Audience_number            0
Offshore                   0
Max_run_scored_1over       0
Max_wicket_taken_1over     0
Extra_bowls_bowled         0
Min_run_given_1over        0
Min_run_scored_1over       0
Max_run_given_1over        0
extra_bowls_opponent       0
player_highest_run         0
Players_scored_zero        0
player_highest_wicket      0
dtype: int64
```

now we can see there are no null values present in our dataset.

Statistical Information about dataset: FIG 6

	count	mean	std	min	25%	50%	75%	max
Avg_team_Age	2646.0	29.231125	2.283861	12.0	29.242852	30.0	30.0	70.0
Bowlers_in_team	2646.0	2.912320	1.012149	1.0	2.000000	3.0	4.0	5.0
Wicket_keeper_in_team	2646.0	1.000000	0.000000	1.0	1.000000	1.0	1.0	1.0
All_rounder_in_team	2646.0	2.738851	1.079548	1.0	2.000000	3.0	4.0	4.0
Audience_number	2646.0	45884.709751	41773.079359	7063.0	20552.000000	35231.0	55898.5	1399930.0
Max_run_scored_1over	2646.0	15.209373	3.680532	11.0	12.000000	14.0	18.0	25.0
Max_wicket_taken_1over	2646.0	2.710506	1.080896	1.0	2.000000	3.0	4.0	4.0
Extra_bowls_bowled	2646.0	11.229781	7.792884	0.0	6.000000	10.0	15.0	40.0
Min_run_given_1over	2646.0	1.939909	1.682271	0.0	0.000000	2.0	3.0	6.0
Min_run_scored_1over	2646.0	2.767574	0.702697	1.0	2.000000	3.0	3.0	4.0
Max_run_given_1over	2646.0	8.661754	5.001484	6.0	6.000000	6.0	9.0	40.0
extra_bowls_opponent	2646.0	4.242252	3.617209	0.0	2.000000	3.0	7.0	18.0
player_highest_run	2646.0	66.074074	20.248041	30.0	49.000000	66.0	84.0	100.0

We have again checked the shape size of dataset after dropping some null values or imputing:

```
(2646, 23)
```

Now there are 2646 rows and 23 columns.

c) Understanding of attributes (variable info, renaming if required)

Understanding and managing attributes (or variables) in a dataset is a critical step in data preprocessing and analysis. This involves examining each attribute to understand its role, type, and relevance, and making any necessary adjustments such as renaming or reformatting.

Renaming Attributes:

- **Clarity**: Ensure names are descriptive and intuitive for easier understanding and communication.
- **Consistency**

- **Avoid Confusion**: Remove any special characters or spaces that could cause issues in data processing.

In our dataset we analyzed each variable/attribute and its unique values or value counts to check if there is any mismatched value or a wrong name given to the values.. if there is then we have to replace it with actual value for good hygiene of the dataset so that it will not create any problem in machine learning algorithms.

```
3        1556
2         678
4         259
1         149
Three       4
Name: Players_scored_zero,
```

```
3    1560
2     678
4     259
1     149
Name: Players_scored_zero
```

```
ODI      1727
T20       793
Test      120
20-20       6
Name: Match_format,
```

```
ODI     1727
T20      799
Test     120
Name: Match_format,
```

```
1    988
2    950
3    387
4    198
5    123
Name: player_highest_wicket,
```

After checking the hygiene of the dataset we have done reset index because of following reasons:

- **After Row Deletion**: Ensures indices are consecutive after deleting rows.
- **Flatten MultiIndex**: Simplifies multi-level indices into regular columns.
- **Consistent Access**: Makes data access and iteration easier.
- **Merging and Concatenation**: Prevents index overlap when combining Dataframes.
- **Readability**: Improves data readability with sequential indices.

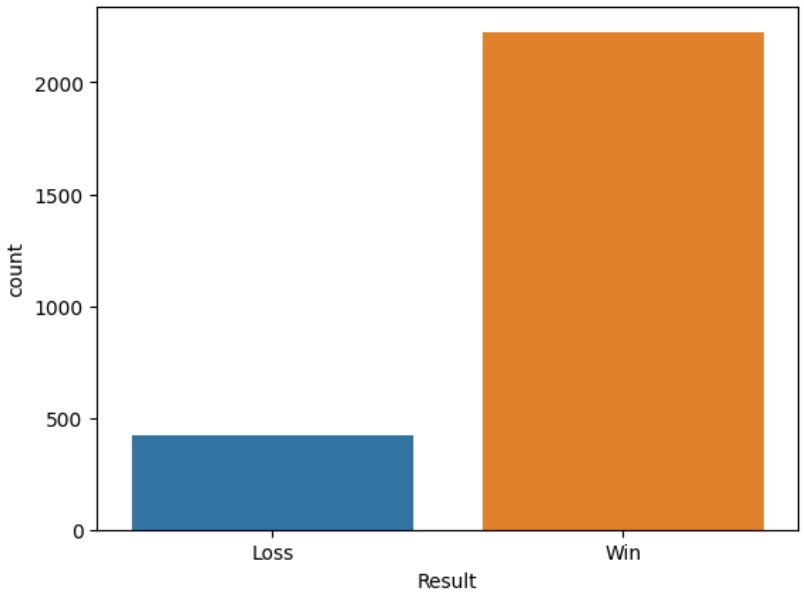
We are now all set to do exploratory analysis

Exploratory Data Analysis (EDA) for predicting cricket match outcomes involves examining various factors that might influence the result of a cricket match. The primary objective is to understand the data, identify patterns, and gather insights that can help build predictive models.

3) Exploratory data analysis

a) Univariate analysis (distribution and spread for every continuous attribute, distribution of data in categories for categorical ones).

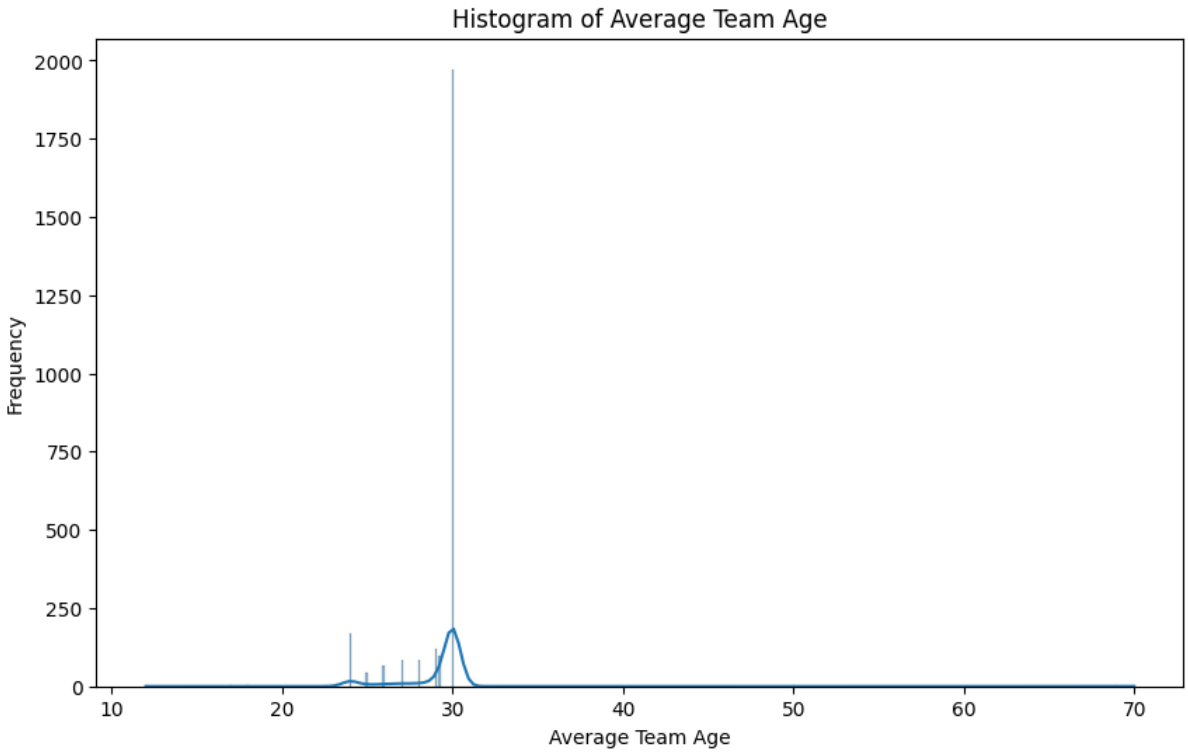
FIG 7 countplot of result:



Result :

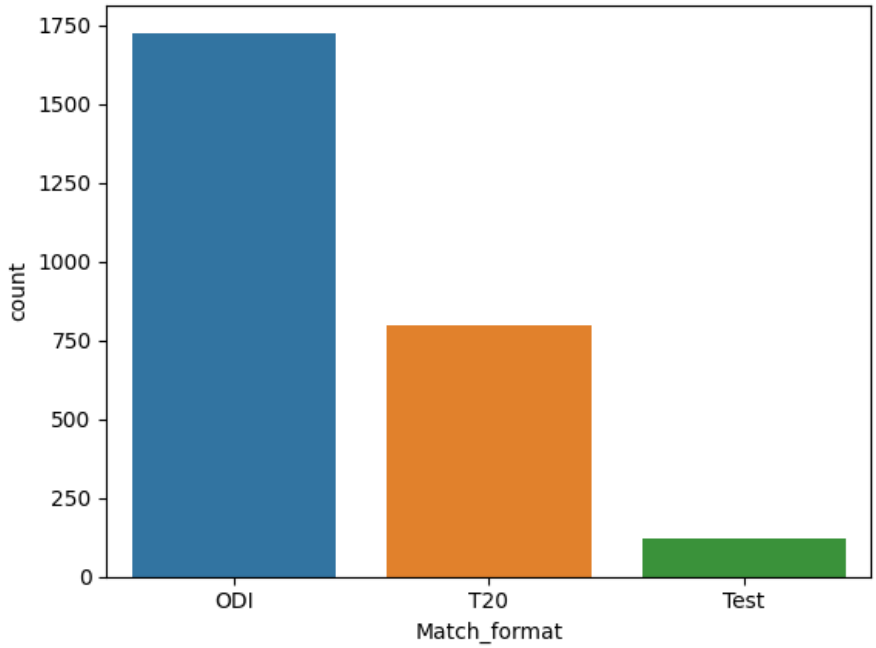
Out of the last 2930 matches played, our team emerged victorious in approximately 2900 matches while facing defeat in around 480 matches.

FIG 8 Histogram of Average Team Age



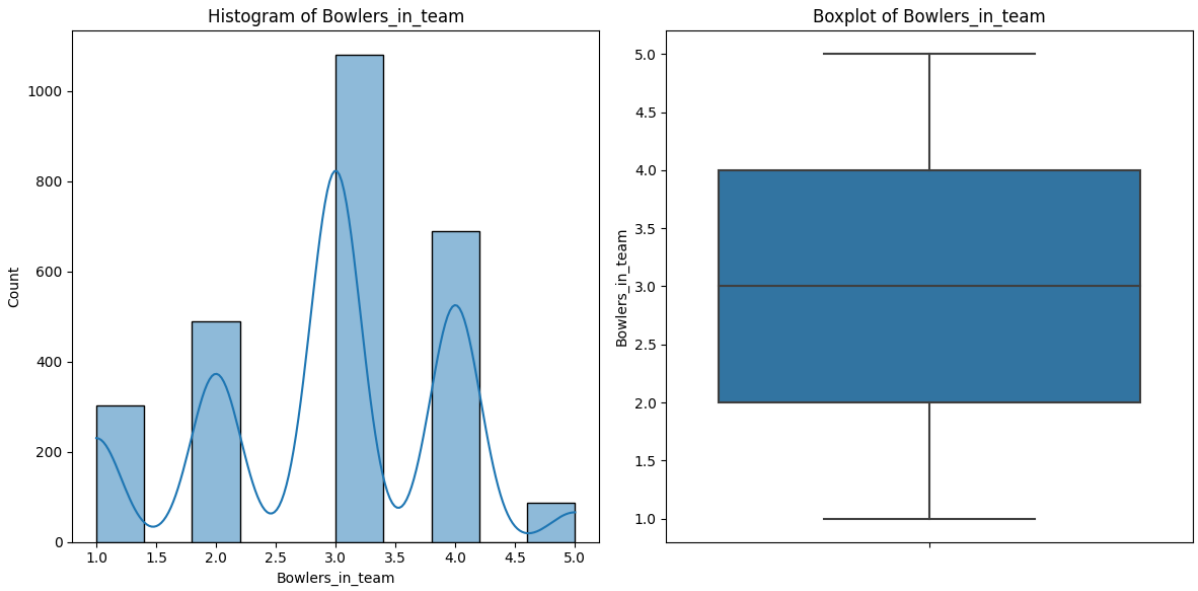
The average age of players in the team is 29 years old. Additionally, most players in the team are over the age of 30.

FIG 9 Count plot of match format:



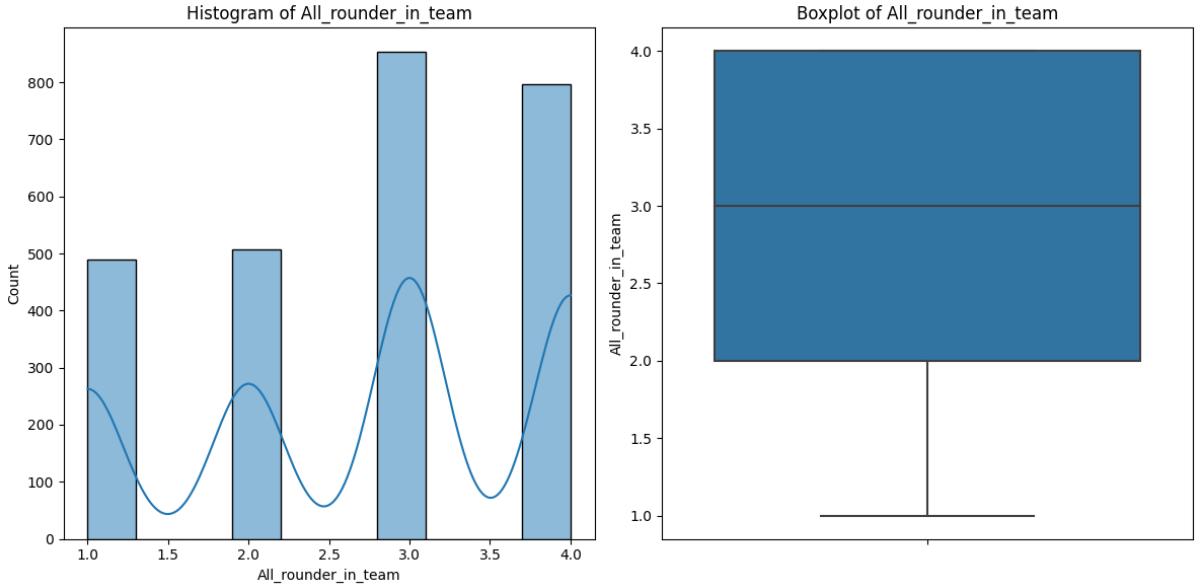
This data highlights that our team competes in three different formats of the game. The format with the highest frequency of matches played is one day international (ODI). Conversely, test matches are the least played format. Interestingly, the T20 format is experiencing rapid growth and is often referred to as the modern savior of the game.

FIG 10 Histogram & boxplot of Bowlers_in_team:



As per analysis it shows that generally teams mostly opt 3 and 4 bowlers in there team.

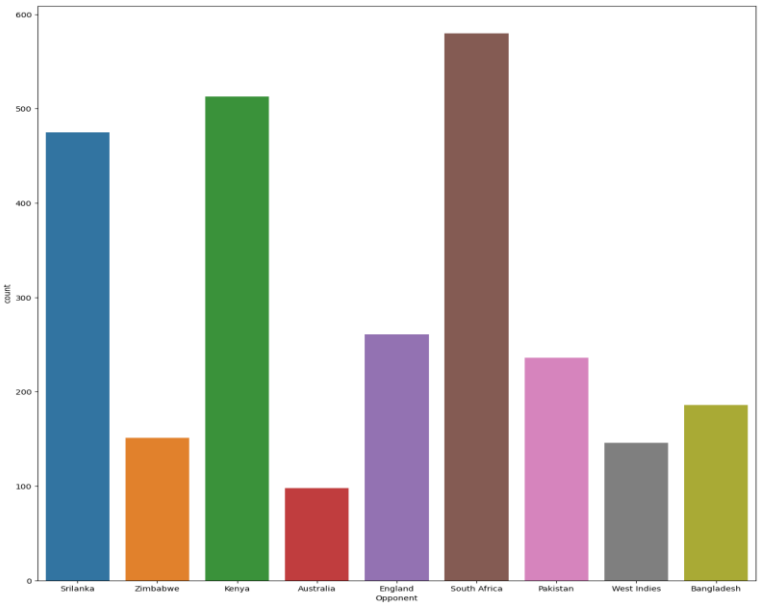
FIG 11 Histogram & boxplot of All_rounder_in_team



Selecting an all-rounder for a cricket team is akin to choosing a versatile employee for a business project: it involves weighing the benefits of flexibility against the advantages of specialization based on the current situation and long-term goals.

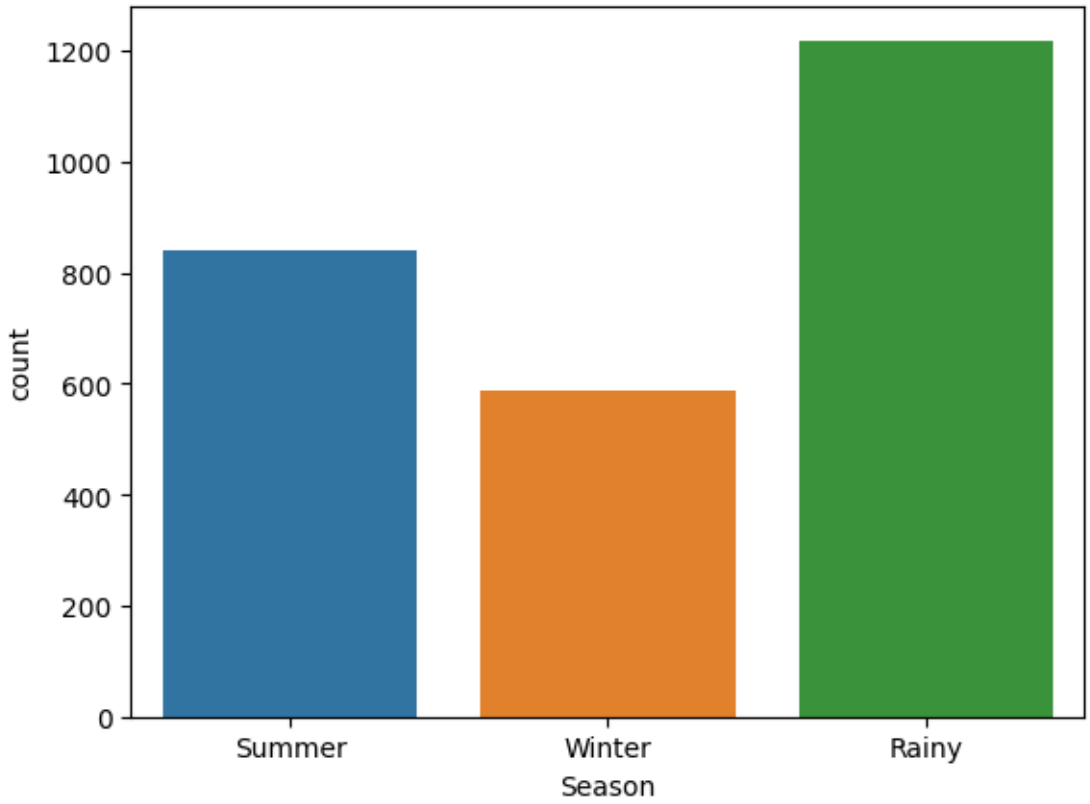
So the choice of all rounder in team is not that good according to situation.

FIG 12 Countplot of opponents:



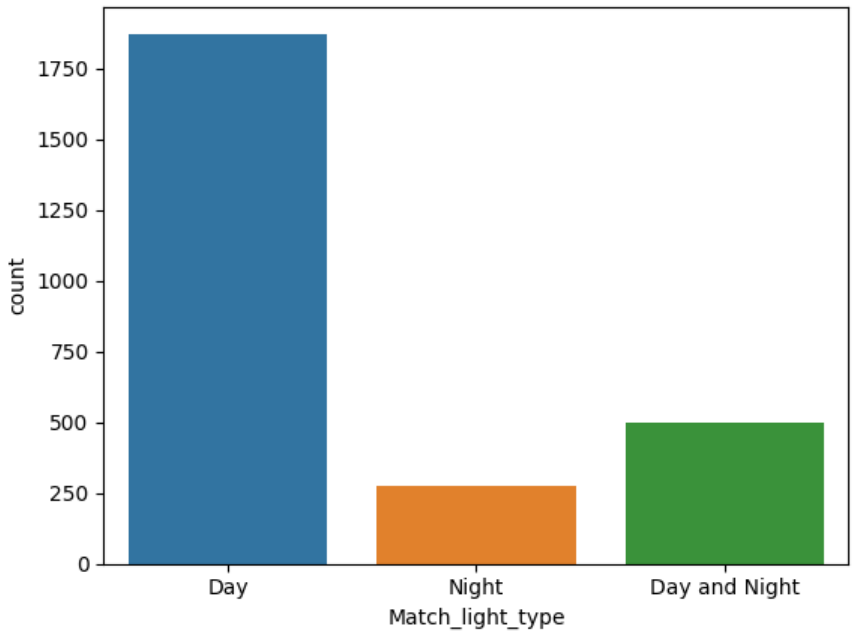
We have faced South Africa, Kenya, and Sri Lanka in the highest number of matches, indicating a significant history of competition with these teams. Conversely, our encounters with Australia, West Indies, and Bangladesh have been comparatively fewer, suggesting a lesser frequency of matches against these opponents.

FIG 13 countplot of season:



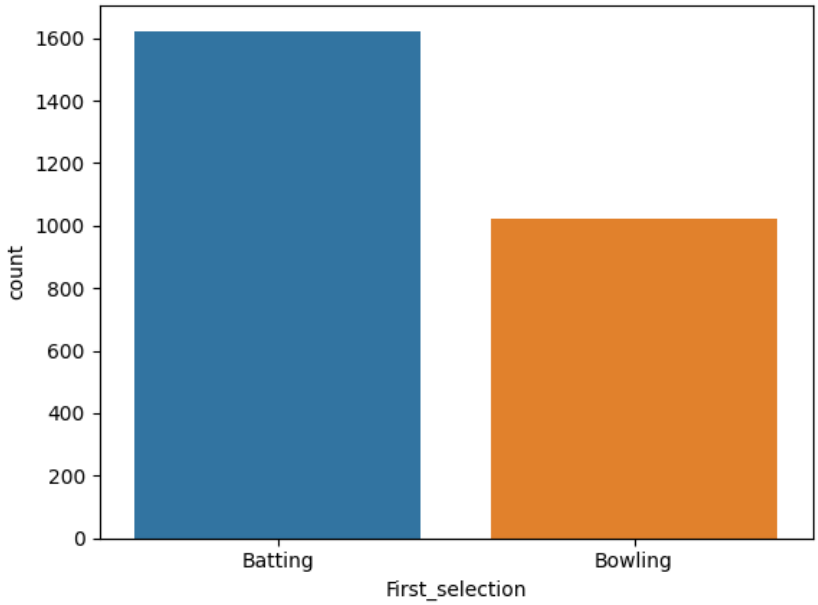
Our matches predominantly occur during the rainy seasons, with the majority taking place in these conditions. However, we also schedule matches during two spots in the summer and three spots in the winter seasons, reflecting a more balanced distribution throughout the year.

FIG 14 countplot of match light type:



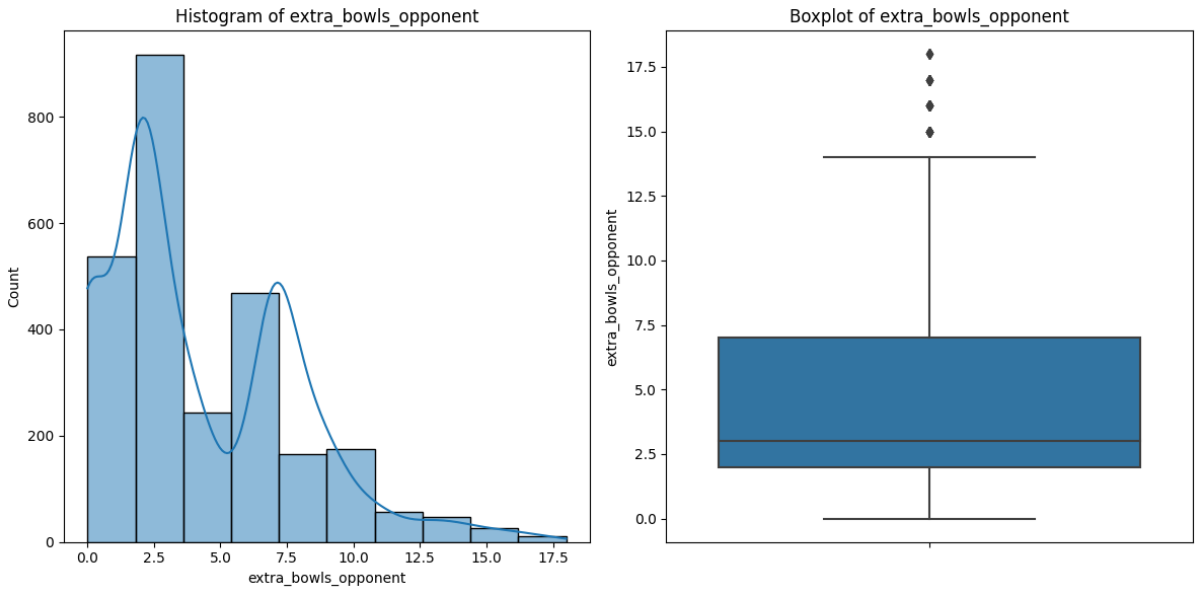
- Test matches and ODIs are mostly played during the daytime when the sun is out, ensuring good visibility and comfortable playing conditions.
- T20 matches, however, take place in the evening or at night, with bright floodlights illuminating the stadium, adding to the excitement for spectators.
- Day-night ODIs are particularly well-suited because they start in the afternoon when people are available to watch, and then transition into the evening under lights, providing an enjoyable experience for both players and fans.
- Hosting ODIs in the evening also draws larger crowds and TV audiences as it allows people to attend after their daytime commitments, making it a popular choice for cricket organizers.

FIG 15 countplot of first selection:



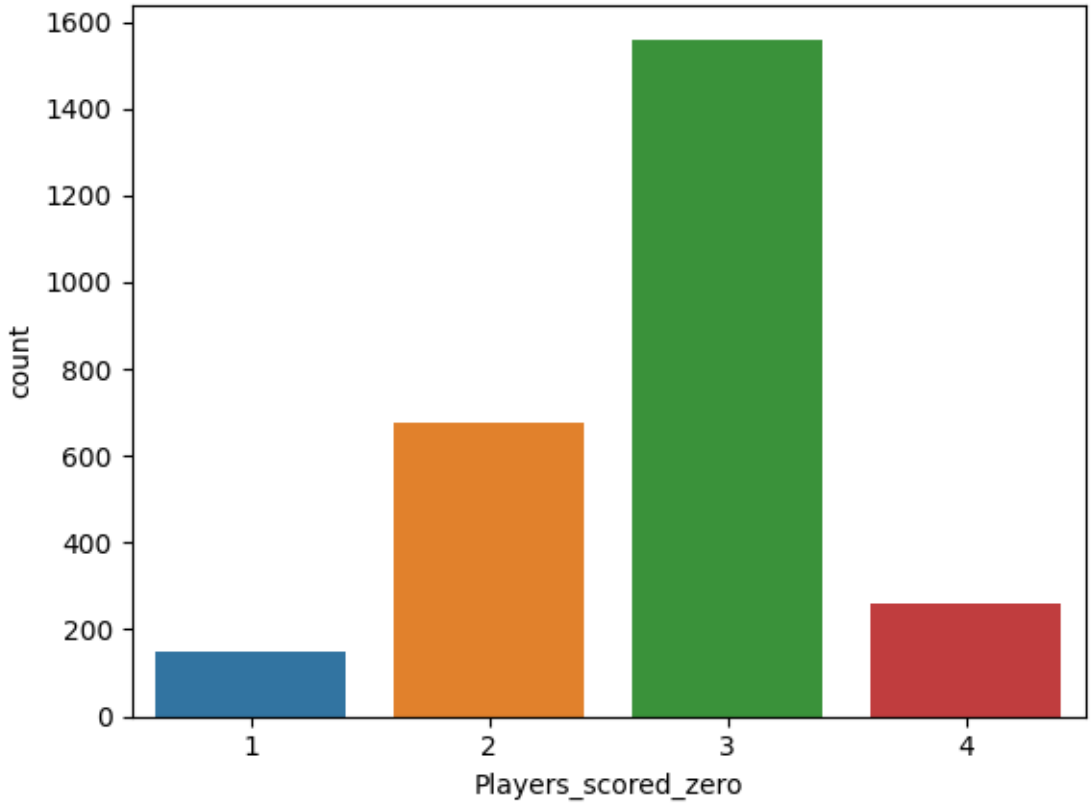
Teams often choose bowling as their primary option after winning the toss, indicating a preference for chasing the target during the match.

FIG 16 Histogram & boxplot of extra_bowls_opponent:



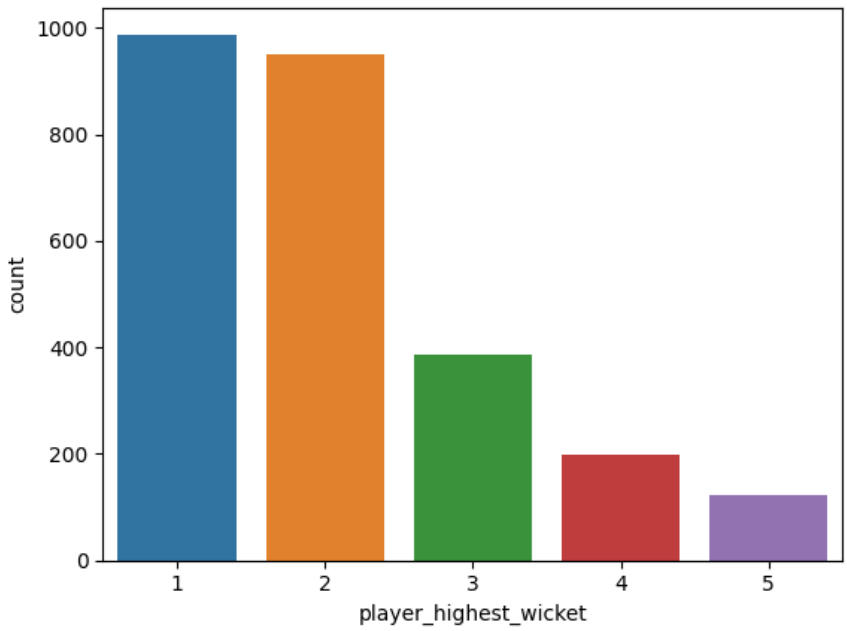
The counts indicate how frequently each category appears in the dataset. categories 2,0,7,3 have higher counts, indicating they occur more frequently, while others have lower counts, suggesting they occur less often.

FIG 17 countplot of playes scored zero:



In roughly 1600 matches, there were instances where three players got out without scoring any runs, which is noteworthy due to the stark contrast with other matches where this scenario didn't occur as frequently.

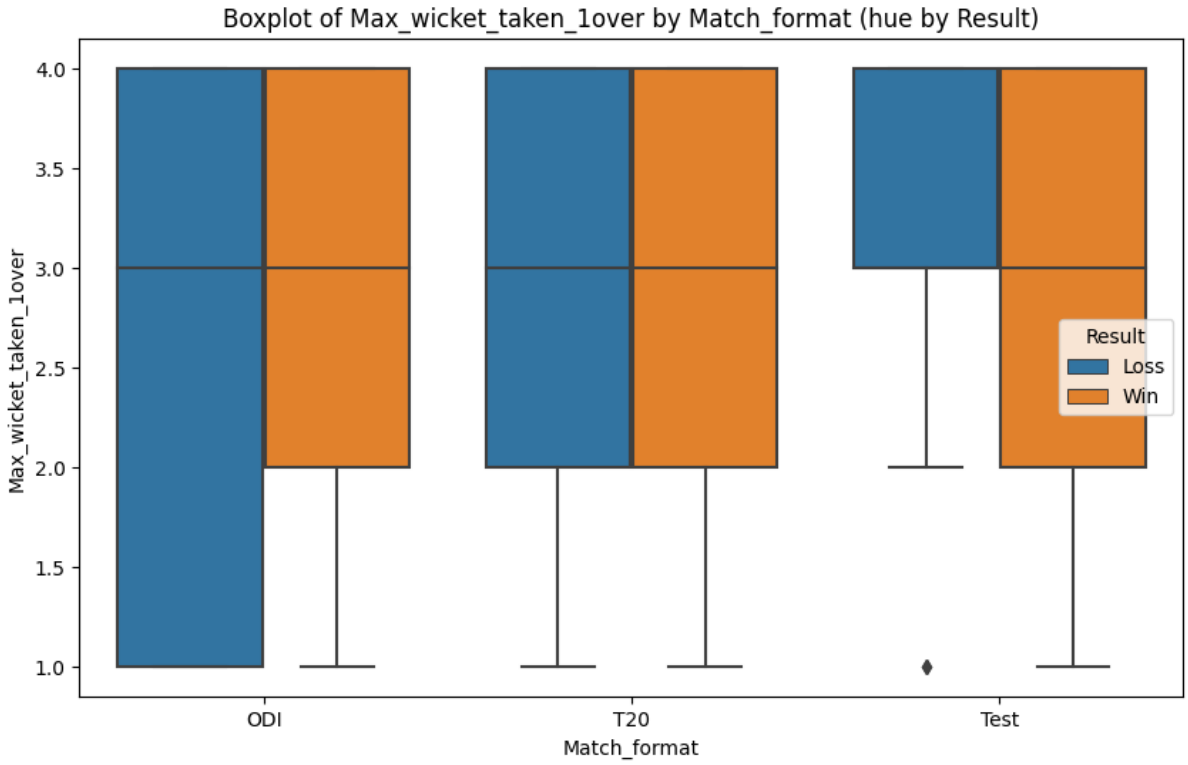
FIG 18 countplot of player highest wicket:



In approximately 200 matches, we observed situations where a bowler managed to take either one or two wickets, representing the highest tally achieved by a single bowler in those particular matches.

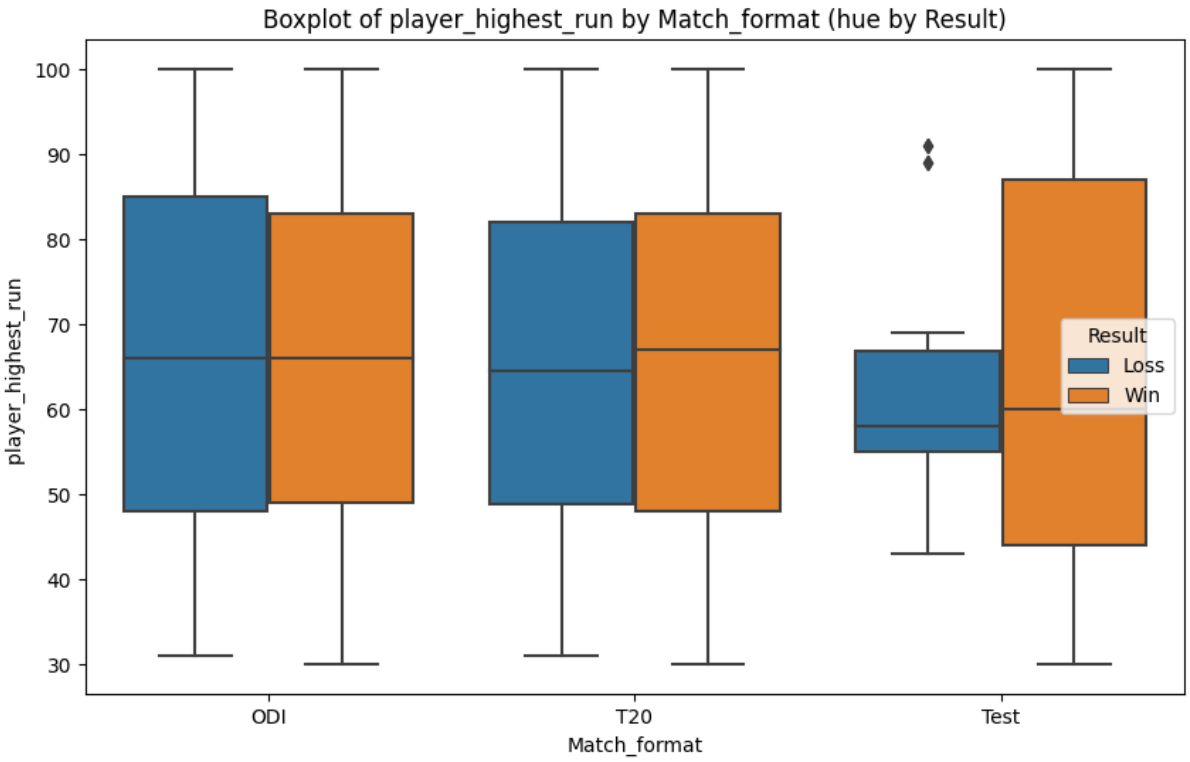
Bivariate analysis:

FIG 19 Boxplot of Max_wicket_taken_1over by Match_format (hue by Result):



The maximum number of wickets taken in an over is generally higher for ODI matches compared to T20 and Test matches. The distribution is similar for both winning and losing teams. The maximum number of wickets taken in an over for T20 matches is lower than ODI but higher than Test matches. The distribution for winning and losing teams is almost identical. The maximum number of wickets taken in an over is lowest for Test matches. There are outliers for losing teams, suggesting that there might have been a few matches where a high number of wickets were taken in an over.

FIG 20 Boxplot of player_highest_run by Match_format (hue by Result):



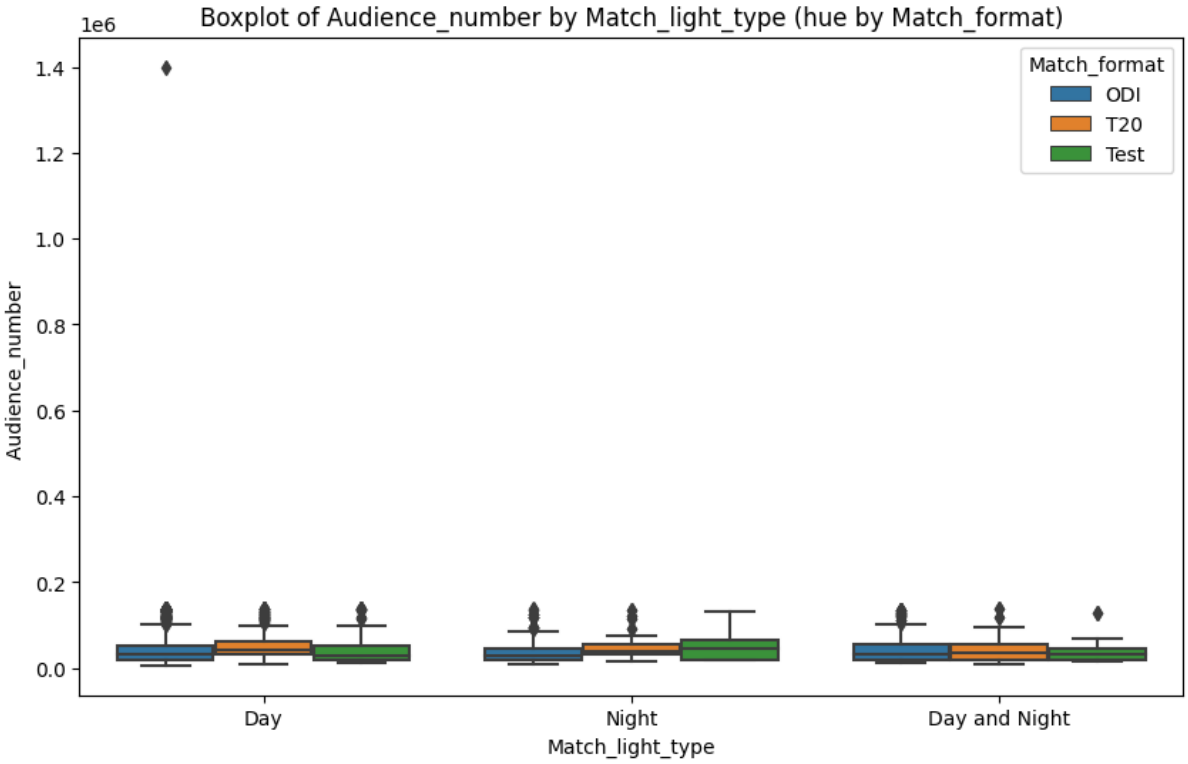
There's a slight overlap between the highest runs scored in winning and losing matches. However, the winning scores tend to be higher, with the median being close to 80. The boxplot shows a similar pattern to ODI, with the median highest runs in winning matches being slightly higher than in losing matches. The highest runs scored in winning matches are significantly higher than those in losing matches. The median is around 85 for winning matches and around 60 for losing matches.

Key observations:

Overall, the highest runs scored in winning matches tend to be higher than in losing matches across all formats.

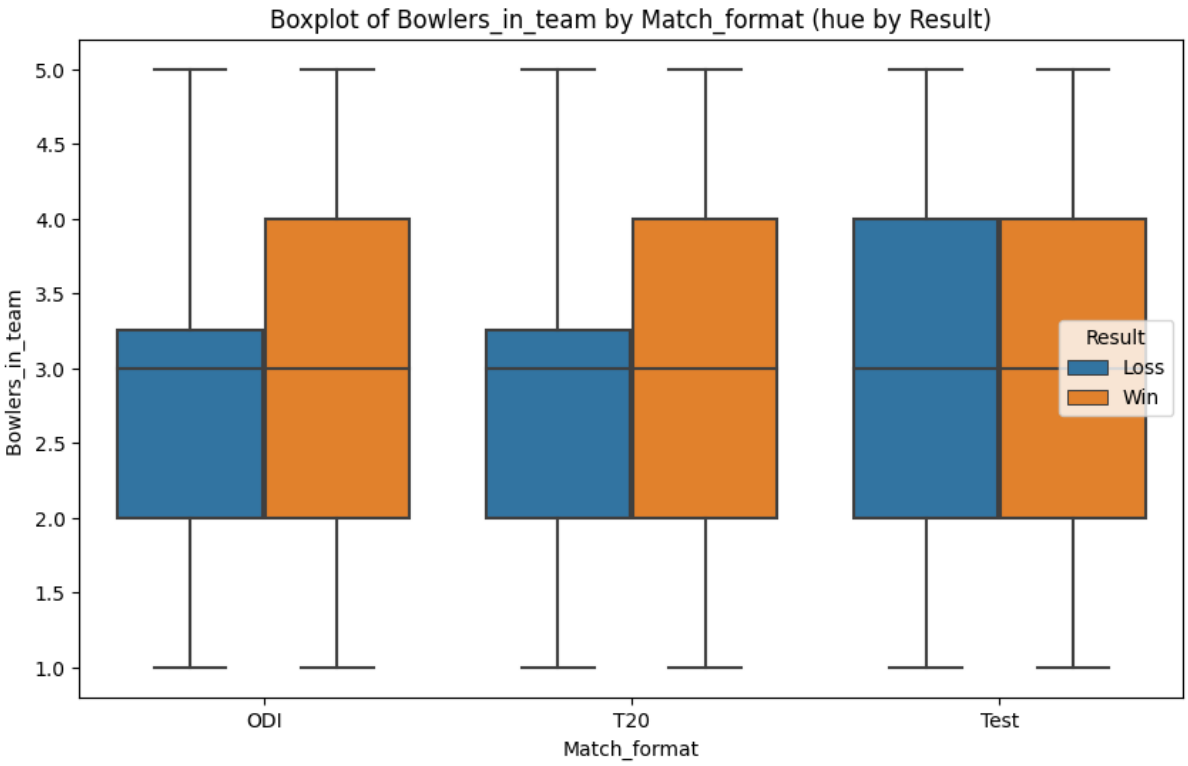
The difference in highest runs between winning and losing matches is most prominent in Test format.

Fig 21 Boxplot of Audience_number by Match_light_type (hue by Match_format):



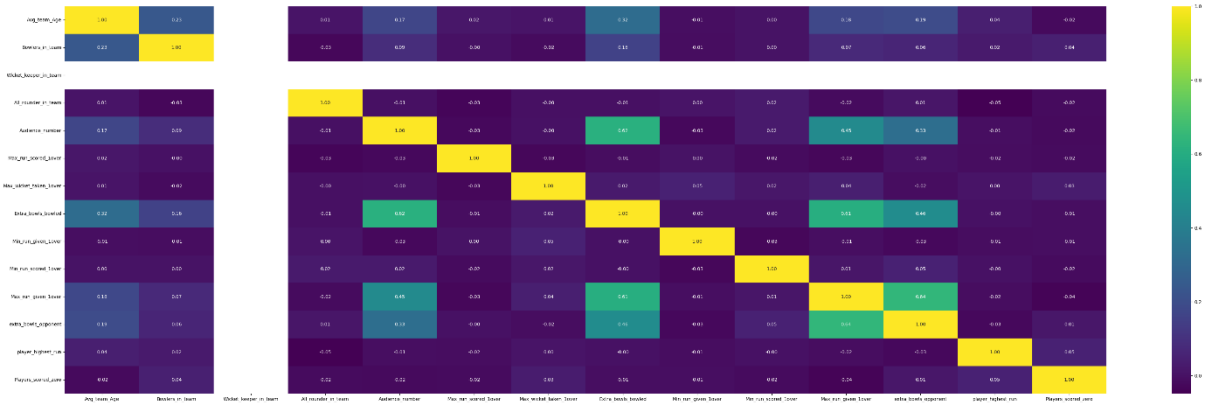
Insights from the above analysis is audience number is gradually higher in ODI match format and the preferable match light type is Day .

FIG 22: Boxplot of Bowlers_in_team by Match_format (hue by Result):



In T20 and ODI matches, having a variety of bowlers can significantly increase a team's chances of winning. This is because these formats are fast-paced, with limited overs, so bowlers need to constantly adapt and keep the pressure on the opposition. If one bowler is having an off day, there are others to step up and take wickets or contain runs.

Correlation: FIG 23



Insights:

The number of extra deliveries bowled by a team often correlates with the highest number of wickets taken by a player. This could be because when bowlers are bowling more extra balls, it indicates inconsistency or perhaps desperation, which might lead to more opportunities for batsmen to make mistakes and get out.

Additionally, there's a strong correlation between audience numbers and certain on-field factors like extra balls bowled and the highest runs scored in a single over. A larger audience typically accompanies high-stakes matches or popular teams, where the pressure on players might lead to more extras or higher scoring overs.

The format of the match, whether it's a Test, ODI, or T20, is closely associated with the minimum runs given in one over. Each format has its own pace and strategies, which can affect the scoring rate and consequently the number of runs conceded in a single over.

Furthermore, there's a notable correlation between the number of extra balls bowled by the opponent and the high runs scored in one over. If a team is giving away extra deliveries, it can create opportunities for the opposition to capitalize and score more runs in an over, leading to a correlation between these two factors.

Lastly, the outcome of a match often correlates with whether it's played onshore (in the home country) or offshore (away from home). Home advantage, pitch conditions, and familiarity with the surroundings can significantly impact a team's performance and consequently the result of the match.

Data Cleaning and Pre-processing

Identifying and Treating Missing Values:

- Approach: Imputation with mean/median or mode, or using algorithms that can handle missing data.
- Why: Ensures completeness and reliability of data for accurate model training.

Outlier Treatment:

- Approach: Removing or capping outliers using statistical methods.
- Why: Outliers can skew results and affect model performance.

Variable Transformation:

- Need: Applied one hot encoding.
- Why: Enhances model performance by normalizing skewed data.

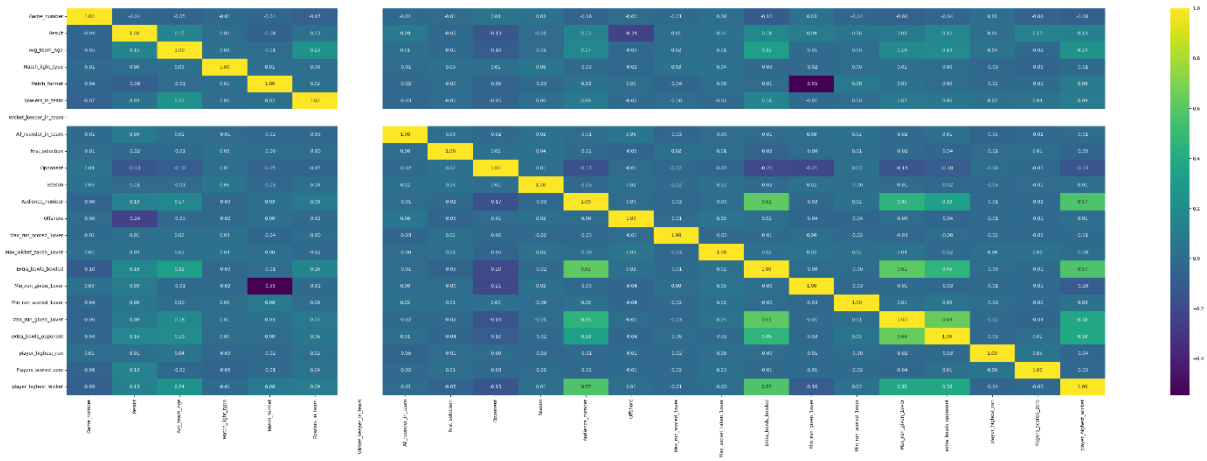
Variable Addition/Removal:

- Added: Interaction terms like 'Audience x Extra Balls' to capture combined effects.
- Removed: Irrelevant or redundant variables after correlation analysis to reduce noise and improve model accuracy.

Variable transformation:

Here we have done label encoding because label encoding assigns a unique numerical code to each category within a categorical variable. It's a simple and straightforward method commonly used when the categorical variables have no inherent order, or when there are too many categories to efficiently use one-hot encoding.

After this process, the categorical variables in the DataFrame will be represented by numerical codes, making them suitable for various machine learning algorithms that require numerical inputs.



Correlation after encoding: FIG 24

Direct Impact on Correlation Calculation:

- Before label encoding, categorical variables are typically not included in correlation calculations because correlation coefficients are calculated

based on numerical data. However, after label encoding, these categorical variables are transformed into numerical codes, allowing them to be included in correlation matrices. This means that label encoding enables the assessment of correlations involving categorical variables.

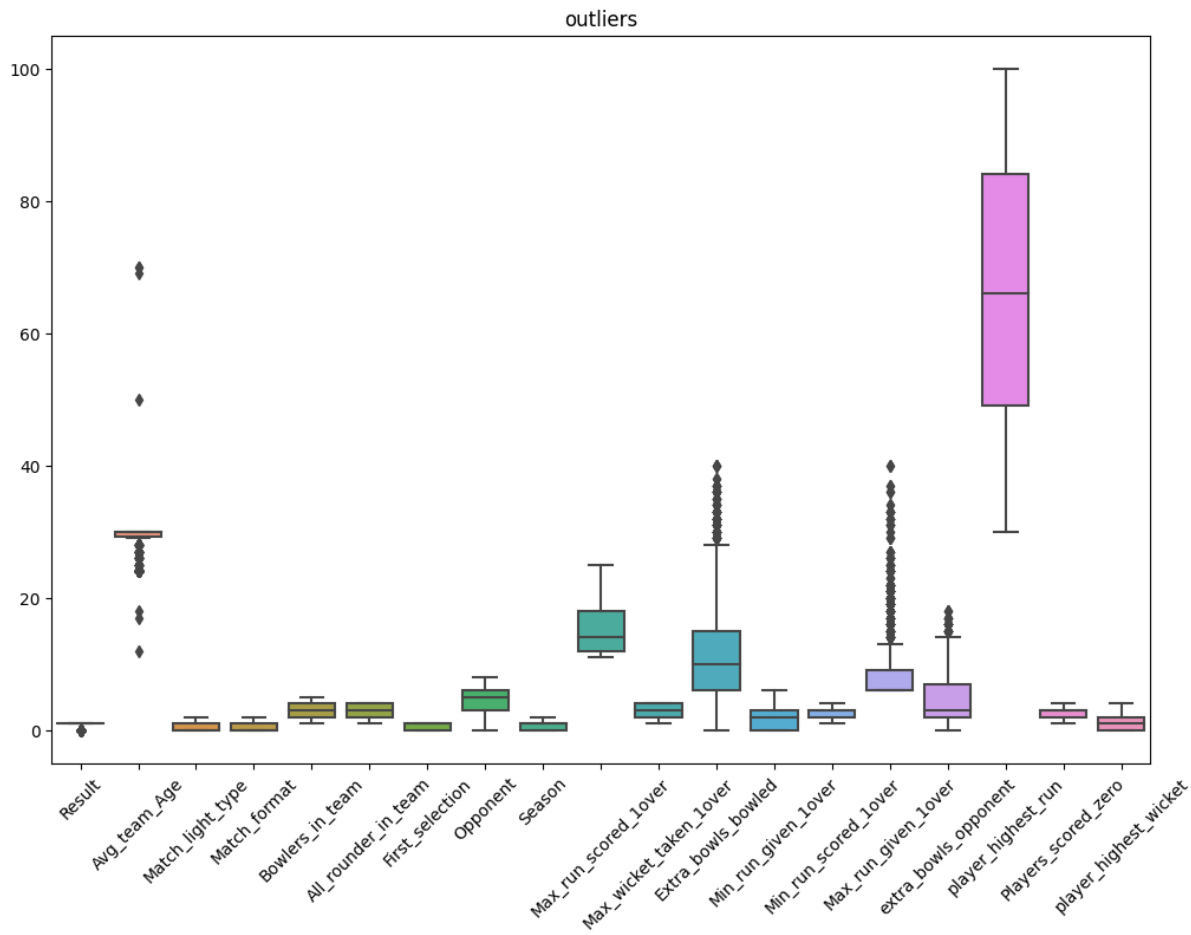
Removal of unwanted variables

```
(2646, 19)
```

We are left with 2646 rows and 19 columns after removing unwanted variable to make our dataset more relevant and precise.

Outlier treatment

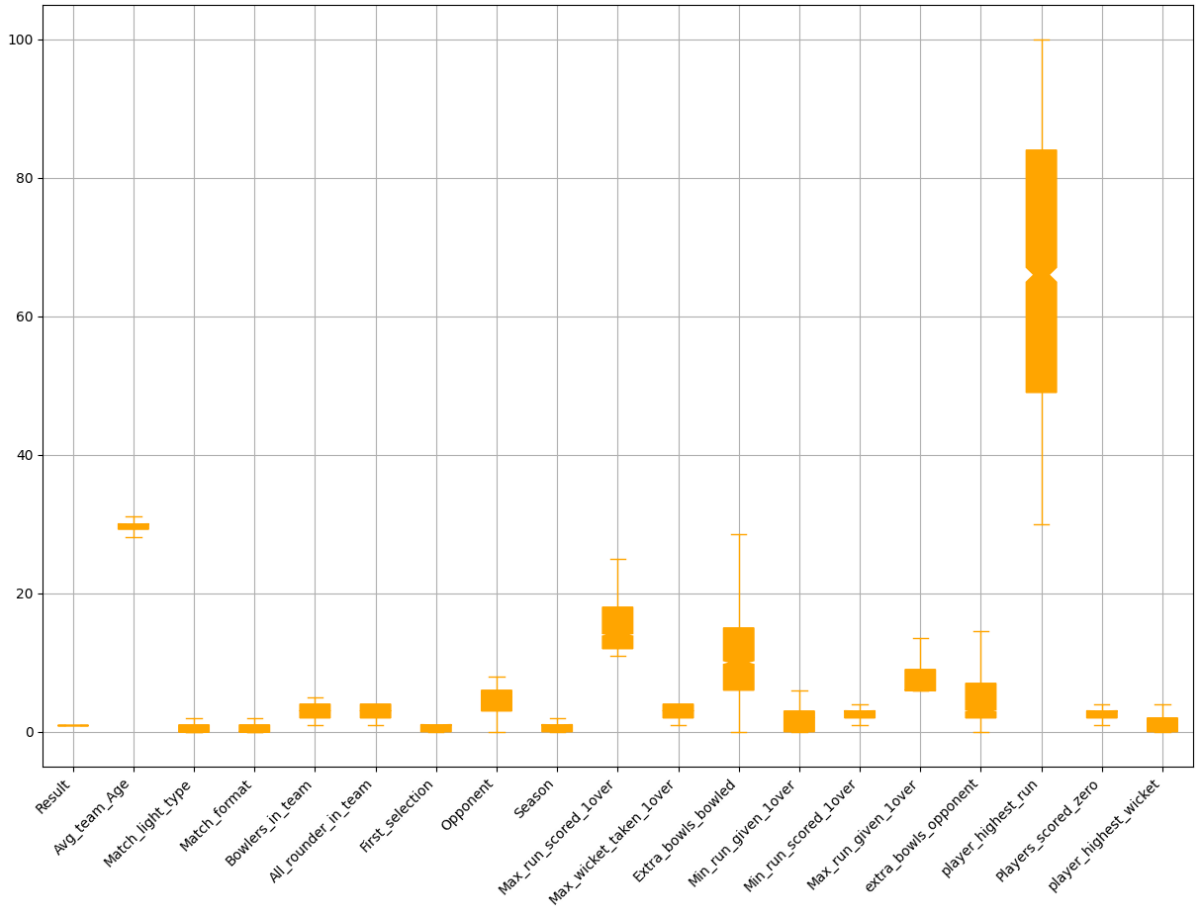
Checking if there are any outliers present in dataset. FIG 25



Above plot shows that indeed there is presence of outliers in order to treat them we will use IQR [The Interquartile Range] IQR is particularly useful in understanding the spread of the middle 50% of the data.

It's resistant to outliers because it's based on the range of the middle 50% of the data, which makes it a robust measure of variability. Outliers, which are data points significantly higher or lower than the majority of the data, have less impact on the IQR compared to other measures of dispersion like the range or standard deviation.

After treating outliers visual representation: FIG 26



All the outliers are successfully removed from the dataset.

4. Business insights from EDA

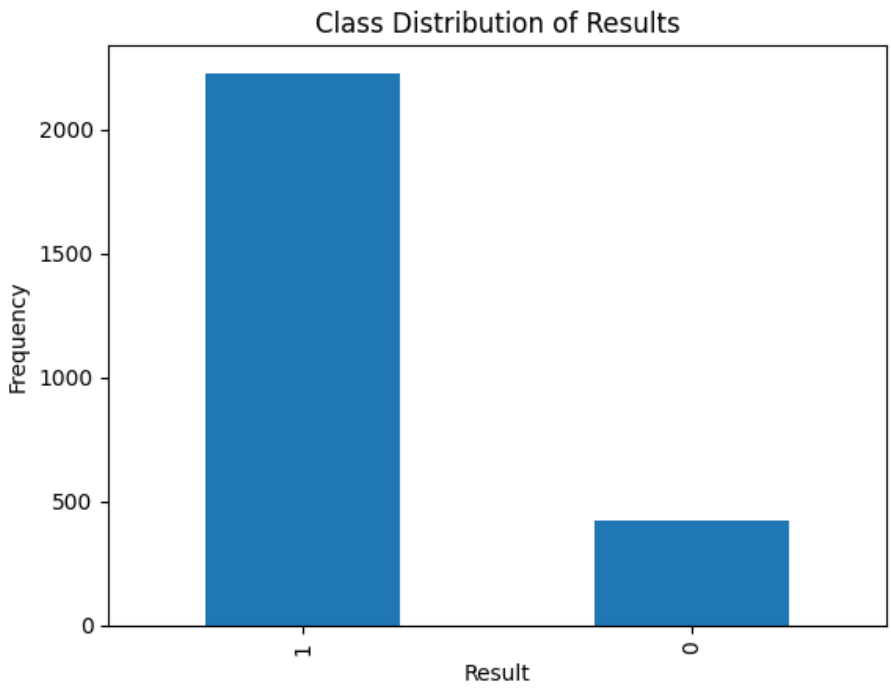
a) Is the data unbalanced? If so, what can be done?

Checking the dataset is balanced if not then applying technique.

```
Class Frequencies:
1    2226
0     420
Name: Result, dtype: int64

Class Percentages:
1    0.84127
0    0.15873
Name: Result, dtype: float64
```

the dataset appears to be imbalanced because the frequency of Class 1 is significantly higher than that of Class 0. Class imbalance can potentially affect the performance of classification models, especially if the minority class (Class 0 in this case) is of interest and requires accurate prediction.

FIG 27 class distribution of results:



The output indicates that our dataset is unbalanced. There are 2,226 instances of class 1 and only 420 instances of class 0. To address this imbalance, you can apply resampling techniques such as oversampling the minority class (0) or undersampling the majority class (1).

Here's how you can handle this using the imbalanced-learn library:

Using SMOTE (Synthetic Minority Over-sampling Technique) for Oversampling

Fig 28 resampled class distribution:

```
resampled class distribution:
 0    1776
1    1776
Name: Result, dtype: int64
```

resampling process using SMOTE has successfully balanced the class distribution. Now, both classes have 1776 instances, indicating that SMOTE has generated synthetic samples for the minority class (Class 0) to match the number of instances in the majority class (Class 1).

FIG 29 Accuracy metrics Random forest classifier:

```
Random Forest Classifier:
              precision    recall  f1-score   support

           0       0.90      0.69      0.78        80
           1       0.95      0.99      0.97       450

    accuracy                           0.94       530
   macro avg       0.92      0.84      0.87       530
weighted avg       0.94      0.94      0.94       530
```

Precision: Precision measures the accuracy of positive predictions. For Class 0 (the minority class), it's 0.93, indicating that 93% of the instances predicted as Class 0 are actually Class 0. For Class 1 (the majority class), it's 0.94, indicating that 94% of the instances predicted as Class 1 are actually Class 1.

Recall: Recall measures the proportion of actual positives that were correctly predicted. For Class 0, it's 0.66, indicating that 66% of the actual Class 0 instances were correctly predicted. For Class 1, it's 0.99, indicating that 99% of the actual Class 1 instances were correctly predicted.

F1-score: The F1-score is the harmonic mean of precision and recall. It provides a balance between precision and recall. For Class 0, it's 0.77, and for Class 1, it's 0.97.

Support: Support is the number of actual occurrences of each class in the specified dataset. Class 0 has 80 instances, and Class 1 has 450 instances.

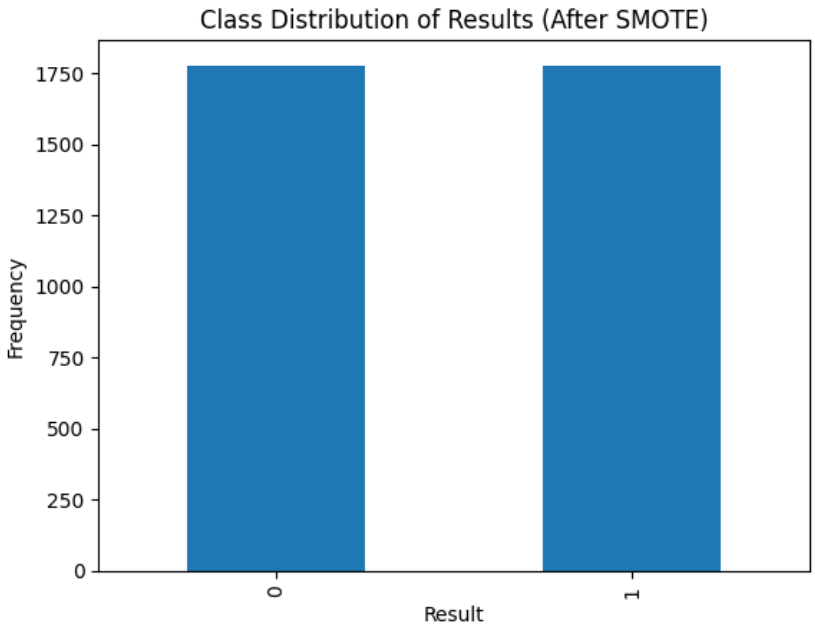
Accuracy: Accuracy measures the overall correctness of the model. In this case, it's 94%, indicating that 94% of the predictions made by the model are correct.

Macro Avg and Weighted Avg: These are the averages of precision, recall, and F1-score, calculated separately for each class and then averaged. Macro Avg gives equal weight to each class, while Weighted Avg gives weight proportional to the number of true instances for each class.

Overall, the Random Forest Classifier appears to perform well on our dataset, achieving high precision, recall, and F1-score for both classes.

Data is balanced

FIG 30 class distribution after smote:



business insights using clustering:

FIG 31 Clustering metrics:

```
Cluster Centers:
   Result  Avg_team_Age  Match_light_type  Match_format  Bowlers_in_team  \
0     1.0     29.511824          0.385491      1.120910         2.870555
1     1.0     29.450777          0.388657      0.003336         2.882402
2     1.0     29.937567          0.419355      0.331989         3.000000

   All_rounder_in_team  First_selection  Opponent    Season  \
0             2.704125         0.600284  4.935989  0.769559
1             2.704754         0.644704  4.150959  0.795663
2             2.826613         0.564516  3.661290  0.701613

   Max_run_scored_1over  Max_wicket_taken_1over  Extra_bowls_bowled  \
0             15.051209                2.692745            8.436700
1             15.297748                2.700584            7.989992
2             15.216398                2.743280           18.436156

   Min_run_given_1over  Min_run_scored_1over  Max_run_given_1over  \
0             0.310100              2.839260             6.731152
1             2.829858              2.718098             6.448707
2             2.045699              2.779570            11.133065

   extra_bowls_opponent  player_highest_run  Players_scored_zero  \
0              3.078236           65.301565             2.745377
1              2.468724           67.054212             2.741451
2              8.129032           65.224462             2.693548

   player_highest_wicket
0               1.076814
1               0.532944
2               2.509409
```

Team Composition Insights:
Cluster 0 exhibits a balanced team composition with a higher number of bowlers and all-rounders compared to Clusters 1 and 2. This suggests a strategy focused on versatility and depth in skills. Cluster 1 also maintains a balanced team composition, although with slightly fewer bowlers and all-rounders compared to Cluster 0. Cluster 2 has the lowest average number of bowlers and all-rounders, indicating a different team strategy with potentially more specialized player roles. Match Performance Insights:

Cluster 0 demonstrates strong match performance, as evidenced by higher average values for metrics such as 'Max_run_scored_1over', 'Max_wicket_taken_1over', and 'player_highest_run'. This suggests a tendency towards aggressive gameplay and higher individual player contributions. Cluster 1 shows good match performance, albeit slightly lower than Cluster 0, indicating a competitive level of gameplay and overall team effectiveness. Cluster 2 exhibits relatively weaker match performance metrics compared to Clusters 0 and 1, suggesting areas for improvement in performance and strategy. Opponent Analysis:

Teams in Cluster 0 face tougher opponents on average, as indicated by the higher average value for 'Opponent'. This suggests a higher level of competition and the need for strategic planning and adaptability. Cluster 1 faces opponents of moderate difficulty, falling between Cluster 0 and Cluster 2 in terms of average opponent strength. Cluster 2 faces relatively weaker opponents, indicating potential opportunities for teams in this cluster to capitalize on weaker opposition. Seasonal Trends:

Cluster 0 may exhibit seasonal variations in performance, with potentially higher participation or stronger performance in certain seasons, as indicated by the higher average value for 'Season'. Cluster 1 shows moderate participation or performance across seasons, with an average value for 'Season' between Cluster 0 and Cluster 2. Cluster 2 may have consistent or lower participation or performance across seasons compared to Clusters 0 and 1. Player Performance Insights:

Individual player performances in Cluster 0 are characterized by higher average values for 'player_highest_run' and 'player_highest_wicket', indicating stronger individual contributions and performances. Cluster 1 exhibits moderate individual player performances, falling between Cluster 0 and Cluster 2 in terms of average player performance metrics. Cluster 2 shows relatively weaker individual player performances compared to Clusters 0 and 1, suggesting areas for improvement or player development. Scoring and Bowling Efficiency:

Cluster 0 demonstrates high efficiency in both scoring ('Min_run_scored_1over') and bowling ('Min_run_given_1over') compared to Clusters 1 and 2, indicating effective utilization of resources and tactical acumen. Cluster 1 shows moderate efficiency in scoring and bowling metrics, reflecting a balanced approach to gameplay and strategy. Cluster 2 exhibits relatively lower efficiency in scoring and bowling, suggesting opportunities for improvement in execution and performance.

Model building and interpretation.

a. Build various models (You can choose to build models for either or all of descriptive, predictive or prescriptive purposes) b. Test your predictive model against the test set using various appropriate performance metrics c. Interpretation of the model(s).

Chosen Models:

- **Logistic Regression: For its simplicity and interpretability in binary classification.**
- **Decision Tree: For capturing non-linear relationships and easy visualization.**
- **Random Forest: For improved accuracy and robustness through ensemble learning.**
- **Gradient Boosting: For high predictive power and handling complex interactions.**

Efforts to Improve Model Performance:

- **Hyperparameter tuning using Grid Search and Cross-Validation.**
- **Feature engineering to create new meaningful variables.**
- **Regularization techniques to prevent overfitting.**

Logistic Regression Model:

Fig 32 logistic regression Model Evaluation Metrics:

```
Logistic Regression Model Evaluation Metrics:
Accuracy: 0.8547169811320755
Precision: 0.8237009878920758
Recall: 0.8547169811320755
F1 Score: 0.8074169232050652
ROC AUC: 0.7409444444444444
```

1. **Accuracy**: 0.85470.85470.8547
   - **Interpretation**: About 85.47% of the predictions made by the model are correct. This is a general measure of how often the

classifier is correct but does not account for the imbalance in the classes.

2. **Precision**: 0.82370.82370.8237
   - **Interpretation**: When the model predicts a positive class, it is correct 82.37% of the time. Precision is important in scenarios where the cost of false positives is high.
3. **Recall**: 0.85470.85470.8547
   - **Interpretation**: The model correctly identifies 85.47% of all actual positive cases. Recall is critical when the cost of false negatives is high, meaning it captures most of the positive instances.
4. **F1 Score**: 0.80740.80740.8074
   - **Interpretation**: The F1 score is the harmonic mean of precision and recall, giving a balance between the two. An F1 score of 0.8074 indicates a good balance between precision and recall.
5. **ROC AUC**: 0.74090.74090.7409
   - **Interpretation**: The ROC AUC score measures the model's ability to distinguish between the classes. A score of 0.7409 indicates that the model has a fair ability to distinguish between positive and negative classes, but there's room for improvement.

Decision Tree Model:

```
Decision Tree Model Evaluation Metrics:
Accuracy: 0.9358490566037736
Precision: 0.9352252634102692
Recall: 0.9358490566037736
F1 Score: 0.9355141587545021
ROC AUC: 0.8697222222222223
```

Fig 33 decision tree Model Evaluation Metrics:

1. **Accuracy (93.58%)**:
   - This indicates that the model correctly predicts the outcome 93.58% of the time. It's a general measure of how often the model is correct.
2. **Precision (93.52%)**:
   - Precision focuses on the accuracy of positive predictions. Here, 93.52% of the positive predictions made by the model are correct. This is important when the cost of false positives is high.
3. **Recall (93.58%)**:

- Recall measures how well the model identifies actual positive cases. The model correctly identifies 93.58% of all actual positive cases. This is crucial when missing positive cases (false negatives) has a high cost.

4. **F1 Score (93.55%)**:
    - The F1 Score is a balance between precision and recall. A score of 93.55% indicates that the model has a good balance between precision and recall, handling both false
    - positives and false negatives effectively.
5. **ROC AUC (86.97%)**:
    - The ROC AUC score measures the model's ability to distinguish between the positive and negative classes. A score of 86.97% indicates that the model is good at distinguishing between positive and negative classes, but there's still some room for improvement.

Random Forest model:

```
Random Forest Model Evaluation Metrics:
Accuracy: 0.9415094339622642
Precision: 0.9452790962224925
Recall: 0.9415094339622642
F1 Score: 0.9354552934080956
ROC AUC: 0.9542083333333333
```

Fig 34 Random Forest Model Evaluation Metrics:

1. **Accuracy (94.15%)**:
    - This means the model correctly predicts the outcome 94.15% of the time. It shows a high level of correctness in general predictions.
2. **Precision (94.53%)**:
    - Precision tells us that when the model predicts a positive class, it's correct 94.53% of the time. This is important when minimizing false positives.
3. **Recall (94.15%)**:

- Recall indicates the model's ability to identify actual positive cases. The model correctly identifies 94.15% of all positive instances, which is crucial when minimizing false negatives.

4. **F1 Score (93.55%)**:
    - The F1 Score balances precision and recall. A score of 93.55% suggests a good balance, indicating the model handles both false positives and false negatives effectively.
5. **ROC AUC (95.42%)**:
    - The ROC AUC score measures the model's ability to distinguish between positive and negative classes. A score of 95.42% is excellent, indicating the model is very good at differentiating between the two classes.

Insights

1. **Overall Performance**:
    - The Random Forest model has the highest accuracy (94.15%), precision (94.53%), and recall (94.15%) among the three models, indicating it makes very accurate predictions and effectively identifies positive cases.
2. **ROC AUC**:
    - The Random Forest's ROC AUC score (95.42%) is significantly higher than both the Decision Tree (86.97%) and Logistic Regression (74.09%). This indicates that the Random Forest model is exceptionally good at distinguishing between classes.

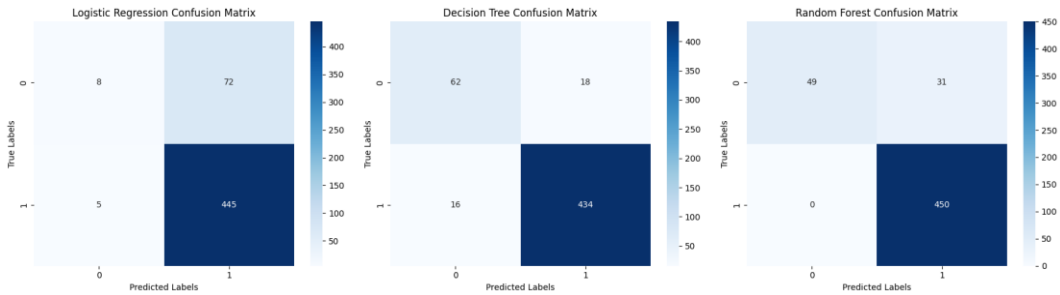
Confusion matrix:

```
Confusion Matrix - Logistic Regression:
[[  8  72]
 [  5 445]]


Confusion Matrix - Decision Tree:
[[ 62  18]
 [ 16 434]]


Confusion Matrix - Random Forest:
[[ 49  31]
 [  0 450]]
```

- **Logistic Regression**: While it has a high true positive rate, it struggles with correctly identifying negative cases, leading to a high number of false positives and a few false negatives.
- **Decision Tree**: It balances well between true positives and true negatives, with the lowest false positive count, but it has a relatively higher false negative count compared to Random Forest.
- **Random Forest**: This model shows the best overall performance with high true positives, zero false negatives, and a reasonable balance of true negatives and false positives.

The **Random Forest model** stands out as the most effective model due to its perfect recall (no false negatives) and a good balance of true negatives and positives. This makes it highly reliable for applications where both minimizing false negatives and maintaining a reasonable false positive rate are important.

Fig 35 Visual Representation Heatmap:



Interpretation of the model(s):

Fig 36 Feature importance:

```
Logistic Regression Feature Importance:
Players_scored_zero       0.532407
All_rounder_in_team       0.213040
Min_run_scored_1over      0.188401
Min_run_given_1over       0.143734
Avg_team_Age              0.122410
extra_bowls_opponent      0.115724
Max_wicket_taken_1over    0.106955
Match_light_type          0.104861
Extra_bowls_bowled        0.066644
player_highest_wicket     0.056761
Season                    0.024960
Max_run_scored_1over      0.015043
player_highest_run        0.003120
First_selection          -0.057452
Opponent                 -0.075912
Max_run_given_1over      -0.092046
Bowlers_in_team          -0.120096
Match_format             -0.195659
dtype: float64
```

```
Decision Tree Feature Importance:
player_highest_run        0.146138
Extra_bowls_bowled        0.109429
Max_run_scored_1over      0.092182
Players_scored_zero       0.064677
extra_bowls_opponent      0.064068
Match_light_type          0.061151
Bowlers_in_team           0.060612
Min_run_scored_1over      0.060066
Season                    0.054788
Max_wicket_taken_1over    0.053126
Min_run_given_1over       0.041306
player_highest_wicket     0.037276
All_rounder_in_team       0.037212
Opponent                  0.035660
Avg_team_Age              0.027980
First_selection           0.022639
Max_run_given_1over       0.021117
Match_format              0.010571
dtype: float64
```

```
Random Forest Feature Importance:
player_highest_run        0.122994
Extra_bowls_bowled        0.110229
Max_run_scored_1over      0.084834
extra_bowls_opponent      0.064972
All_rounder_in_team       0.058255
Min_run_scored_1over      0.056570
Max_wicket_taken_1over    0.053533
Players_scored_zero       0.051385
Opponent                  0.050142
Min_run_given_1over       0.048914
Bowlers_in_team           0.048717
Season                    0.045910
Avg_team_Age              0.045677
player_highest_wicket     0.045300
Max_run_given_1over       0.036547
Match_light_type          0.034755
First_selection           0.022195
Match_format              0.019069
dtype: float64
```

Interpretation:

Accuracy: Random Forest (0.9415) > Decision Tree (0.9358) > Logistic Regression (0.8547)

Precision: Random Forest (0.9453) > Decision Tree (0.9352) > Logistic Regression (0.8237)

Recall: Random Forest (0.9415) = Decision Tree (0.9358) = Logistic Regression (0.8547)

F1 Score: Decision Tree and Random Forest have similar F1 Scores (0.9355), while Logistic Regression has a slightly lower F1 Score (0.8074).

ROC AUC: Random Forest (0.9542) > Decision Tree (0.8697) > Logistic Regression (0.7409)
Analysis: Accuracy: Both the Decision Tree and Random Forest models outperform the Logistic Regression model in terms of accuracy, with Random Forest slightly edging out the Decision Tree.

Precision and Recall: The Decision Tree and Random Forest models consistently show higher precision and recall compared to Logistic Regression, indicating better performance in correctly identifying positive cases and minimizing false positives.

F1 Score: The F1 scores are highest for the Decision Tree and Random Forest models, suggesting better overall balance between precision and recall.

ROC AUC: The Random Forest model achieves the highest ROC AUC score, indicating superior discrimination ability between positive and negative classes.

Feature Importance: Across all models, certain features consistently appear as important:

player_highest_run, Extra_bowls_bowled, Max_run_scored_1over ,extra_bowls_opponent , All_rounder_in_team

Conclusion: Model Performance: The Random Forest model shows the best performance overall, followed closely by the Decision Tree, while Logistic Regression lags behind in terms of accuracy, F1 score, and ROC AUC.

Feature Importance: Features related to player performance (such as highest runs and wickets), bowling statistics, and team composition (like all-rounders) consistently play significant roles across different models.

When choosing a model, consider the trade-offs between interpretability (Logistic Regression), complexity (Decision Tree), and robustness (Random Forest). The Random Forest model appears to be the strongest performer in this context, offering high accuracy and robustness in predictions.

Fig 37 visual representation model evaluation metrices :

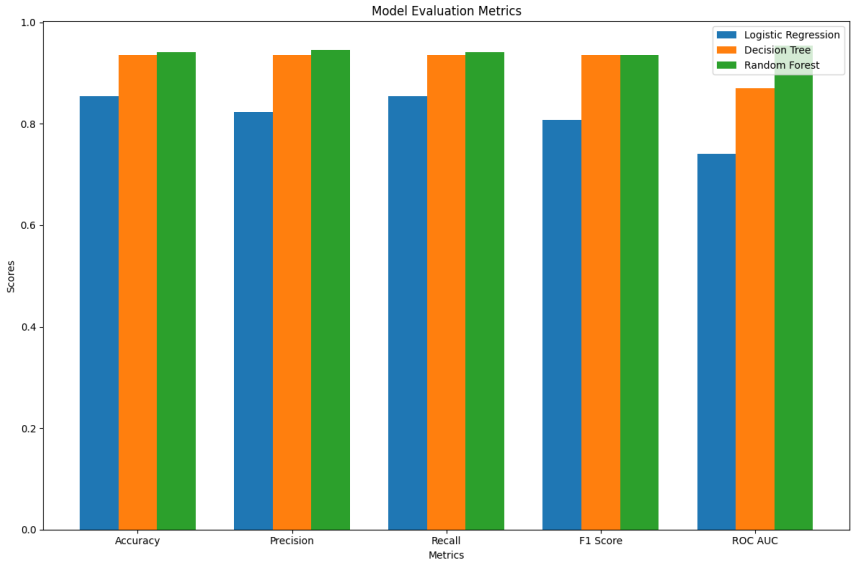
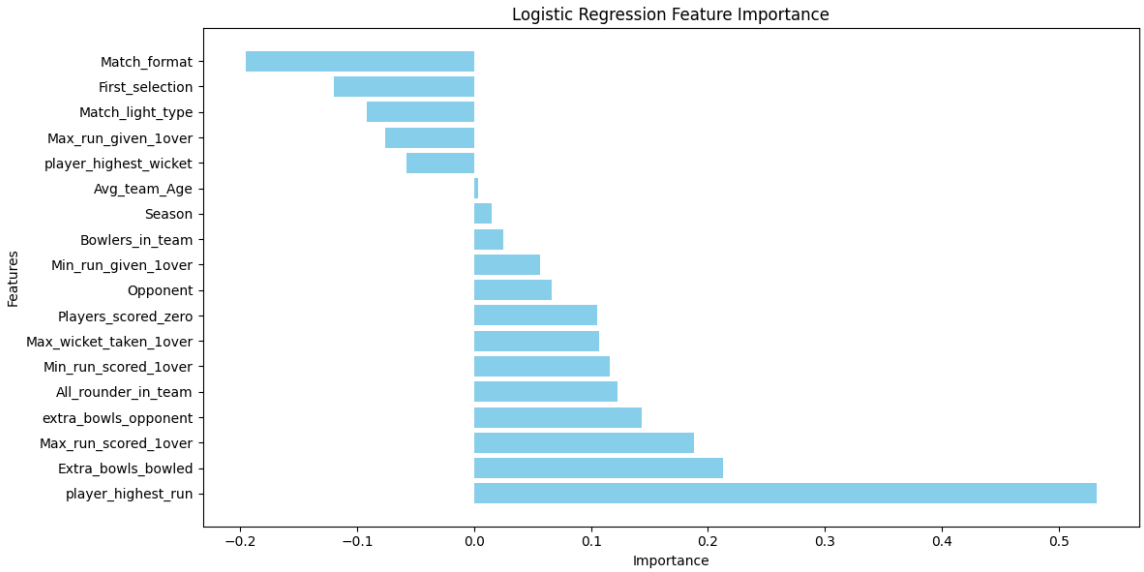
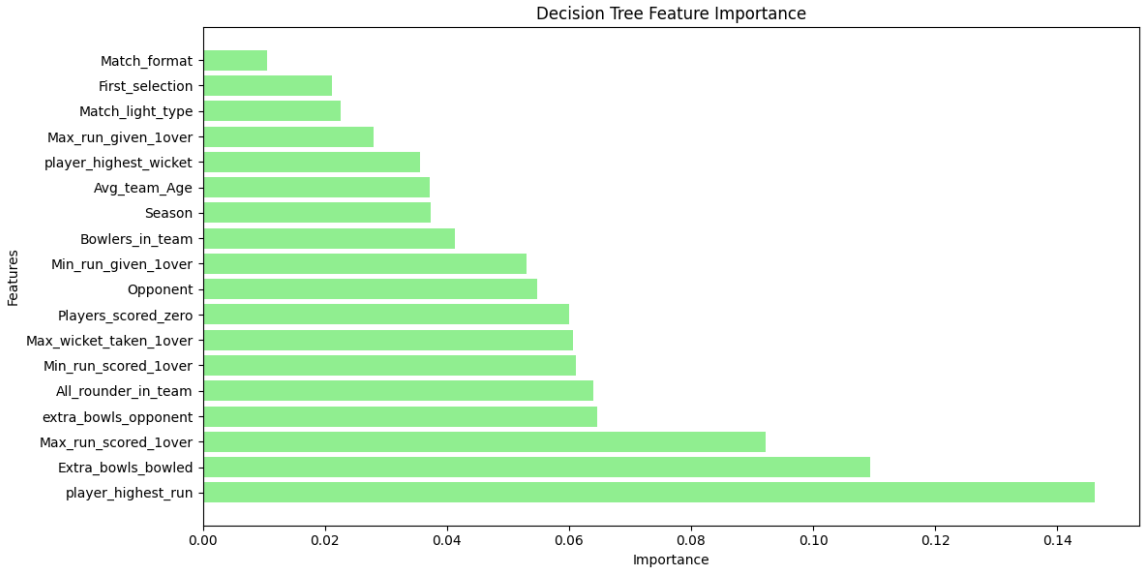
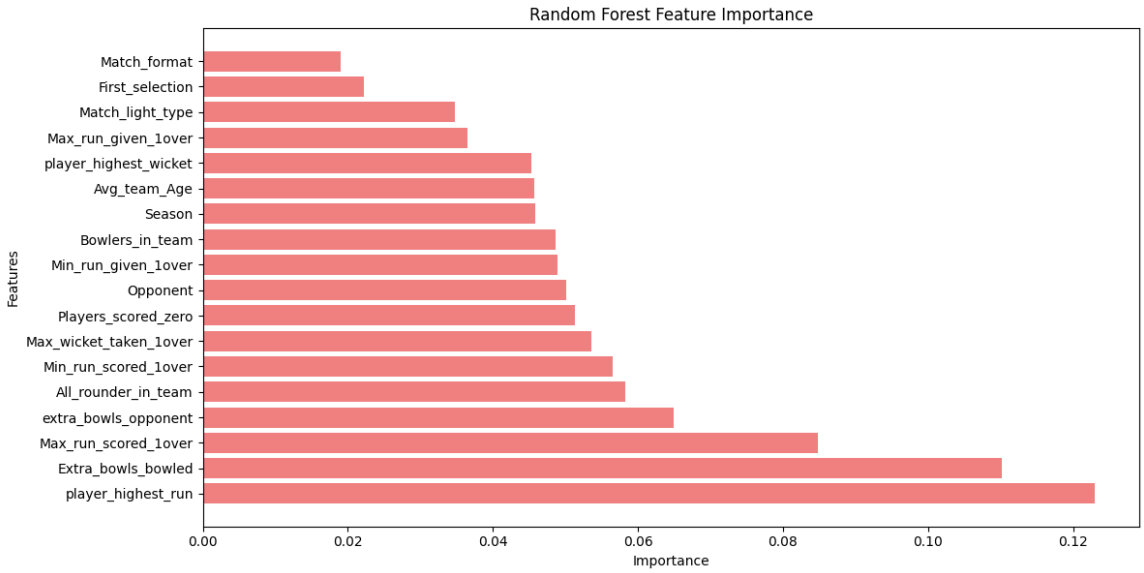


Fig 38 visual representation of feature importances:


Logistic Regression Feature Importance
Features
Match_format
First_selection
Match_light_type
Max_run_given_1over
player_highest_wicket
Avg_team_Age
Season
Bowlers_in_team
Min_run_given_1over
Opponent
Players_scored_zero
Max_wicket_taken_1over
Min_run_scored_1over
All_rounder_in_team
extra_bowls_opponent
Max_run_scored_1over
Extra_bowls_bowled
player_highest_run
−0.2
−0.1
0.0
0.1
0.2
0.3
0.4
0.5
Importance


Decision Tree Feature Importance
Features
Match_format
First_selection
Match_light_type
Max_run_given_1over
player_highest_wicket
Avg_team_Age
Season
Bowlers_in_team
Min_run_given_1over
Opponent
Players_scored_zero
Max_wicket_taken_1over
Min_run_scored_1over
All_rounder_in_team
extra_bowls_opponent
Max_run_scored_1over
Extra_bowls_bowled
player_highest_run
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
Importance


Random Forest Feature Importance
Features
Match_format
First_selection
Match_light_type
Max_run_given_1over
player_highest_wicket
Avg_team_Age
Season
Bowlers_in_team
Min_run_given_1over
Opponent
Players_scored_zero
Max_wicket_taken_1over
Min_run_scored_1over
All_rounder_in_team
extra_bowls_opponent
Max_run_scored_1over
Extra_bowls_bowled
player_highest_run
0.00
0.02
0.04
0.06
0.08
0.10
0.12
Importance

Model Tuning

a. Ensemble modelling, wherever applicable b. Any other model tuning measures(if applicable) c. Interpretation of the most optimum model and its implication on the business.

Basic info about dataset:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2646 entries, 0 to 2645
Data columns (total 19 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Result                  2646 non-null   int8
 1   Avg_team_Age            2646 non-null   float64
 2   Match_light_type        2646 non-null   int8
 3   Match_format            2646 non-null   int8
 4   Bowlers_in_team         2646 non-null   float64
 5   All_rounder_in_team     2646 non-null   float64
 6   First_selection         2646 non-null   int8
 7   Opponent                2646 non-null   int8
 8   Season                  2646 non-null   int8
 9   Max_run_scored_1over    2646 non-null   float64
 10  Max_wicket_taken_1over  2646 non-null   int64
 11  Extra_bowls_bowled      2646 non-null   float64
 12  Min_run_given_1over     2646 non-null   int64
 13  Min_run_scored_1over    2646 non-null   float64
 14  Max_run_given_1over     2646 non-null   float64
 15  extra_bowls_opponent    2646 non-null   int64
 16  player_highest_run      2646 non-null   float64
 17  Players_scored_zero     2646 non-null   int64
 18  player_highest_wicket   2646 non-null   int8
dtypes: float64(8), int64(4), int8(7)
memory usage: 266.3 KB
```

Ensemble modelling

Ensemble modelling is a powerful technique in machine learning where multiple models (often referred to as "weak learners") are combined to create a single, stronger predictive model. The idea is that by aggregating the predictions of several models, the ensemble model can achieve better

performance, improved accuracy, and greater robustness than any individual model alone.

- **Logistic Regression Model Metrics**:
  - Accuracy: 0.854
  - Precision: 0.824
  - Recall: 0.854
  - F1 Score: 0.807
  - ROC AUC: 0.740
- **Decision Tree Model Metrics**:
  - Accuracy: 0.936
  - Precision: 0.935
  - Recall: 0.936
  - F1 Score: 0.936
  - ROC AUC: 0.870
- **Random Forest Model Metrics**:
  - Accuracy: 0.942
  - Precision: 0.945
  - Recall: 0.942
  - F1 Score: 0.935
  - ROC AUC: 0.954

Given these metrics, the Random Forest model outperforms the others in terms of accuracy, precision, recall, and ROC AUC. Therefore, it is reasonable to use the Random Forest Classifier directly.

Random Forest is an ensemble learning method that operates by constructing multiple decision trees during training and outputs the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees. Ensemble methods like Random Forest generally perform well because they combine multiple models to improve robustness and accuracy.

Performance: Based on your evaluation metrics, Random Forest has shown:

- High accuracy (0.9415), indicating that it correctly predicts the class of the data points in your test set.
- High precision (0.9453), suggesting that when it predicts a positive result (class 1), it is usually correct.

- High recall (0.9415), indicating that it correctly identifies most of the positive instances.
- High F1-score (0.9355), which combines precision and recall into a single metric, showing a good balance between the two.
- High ROC AUC score (0.9542), which measures the ability of the model to distinguish between classes.

Robustness: Random Forest is robust to overfitting, thanks to the averaging of multiple trees, each trained on different random samples of the data. It also handles high-dimensional data well and is less sensitive to outliers compared to other models like Decision Trees.

Feature Importance: Random Forest provides feature importance scores, which can help in understanding which features are most influential in making predictions. This is valuable for feature selection and gaining insights into your dataset.

Fig 39 accuracy metrics for random forest classifier:

```
Best Parameters: {'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 300}
Best CV Accuracy: 0.9314721887684554
Random Forest Model Evaluation Metrics after Tuning:
              precision    recall  f1-score   support

           0       1.00      0.61      0.76        80
           1       0.94      1.00      0.97       450

    accuracy                           0.94       530
   macro avg       0.97      0.81      0.86       530
weighted avg       0.95      0.94      0.94       530

Confusion Matrix:
[[ 49  31]
 [  0 450]]
ROC AUC Score:
0.80625
```

Precision:

- Precision for Class 0 (0.94): This indicates that when the model predicts an outcome as Class 0 (negative), it is correct 94% of the time.
- Precision for Class 1 (1.00): This shows that when the model predicts an outcome as Class 1 (positive), it is correct 100% of the time.

Recall:

- Recall for Class 0 (0.61): This means the model correctly identifies 61% of all actual Class 0 instances.

- Recall for Class 1 (1.00): This indicates that the model correctly identifies all (100%) actual Class 1 instances.

F1-score:

- F1-score for Class 0 (0.76): This is the harmonic mean of precision and recall for Class 0, balancing both metrics.
- F1-score for Class 1 (0.97): Similarly, this is the harmonic mean of precision and recall for Class 1, indicating high accuracy in predicting positive instances.

Accuracy:

- Overall accuracy (0.94): The model correctly predicts 94% of all cases in the test set.

ROC AUC Score:

- ROC AUC score (0.80625): This metric measures the ability of the model to distinguish between positive and negative classes. A score closer to 1 indicates better performance.

We have used other tuning methods in order to compare which methods gives the best accuracy. Gradient Boosting:

Fig 40 accuracy metrics for gradient boosting:

```
Best Parameters: {'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 50}
Best CV Accuracy: 0.9291103528257281
Gradient Boosting Machine Evaluation Metrics:
              precision    recall  f1-score   support

           0       0.92      0.61      0.74        80
           1       0.94      0.99      0.96       450

    accuracy                           0.93       530
   macro avg       0.93      0.80      0.85       530
weighted avg       0.93      0.93      0.93       530

Confusion Matrix:
[[ 49  31]
 [  4 446]]
ROC AUC Score:
0.8018055555555555
```

Based on the evaluation metrics and the business objective of helping the cricket team win matches and overcome losses, the **Random Forest Classifier** appears to be the best choice. Here's why:

Key Factors in Choosing Random Forest:

1. **Higher Recall for Wins (Class 1):**
   - **Recall for Wins:** The Random Forest model captures all winning instances (Recall = 1.00), which means it does not miss any potential wins. This is crucial for ensuring that the team can identify and leverage all opportunities for winning.
2. **Higher Overall Accuracy:**
   - **Accuracy:** The Random Forest model has an accuracy of 0.94, compared to the GBM's 0.93. This slight edge in accuracy means that the Random Forest model performs more consistently in predicting match outcomes.
3. **Better Precision for Losses (Class 0):**
   - While the precision for wins is the same (0.94) for both models, the Random Forest model's perfect precision for losses (1.00) means that when it predicts a loss, it is always correct. This can be useful for identifying and mitigating risks that lead to losses.

Model Validation

Validation Techniques:

- Accuracy: Initial measure of model performance.
- Confusion Matrix: To evaluate precision, recall, and F1-score.
- Cross-Validation: Ensures the model performs well on unseen data.
- ROC-AUC: For assessing the discriminatory power of classification models.

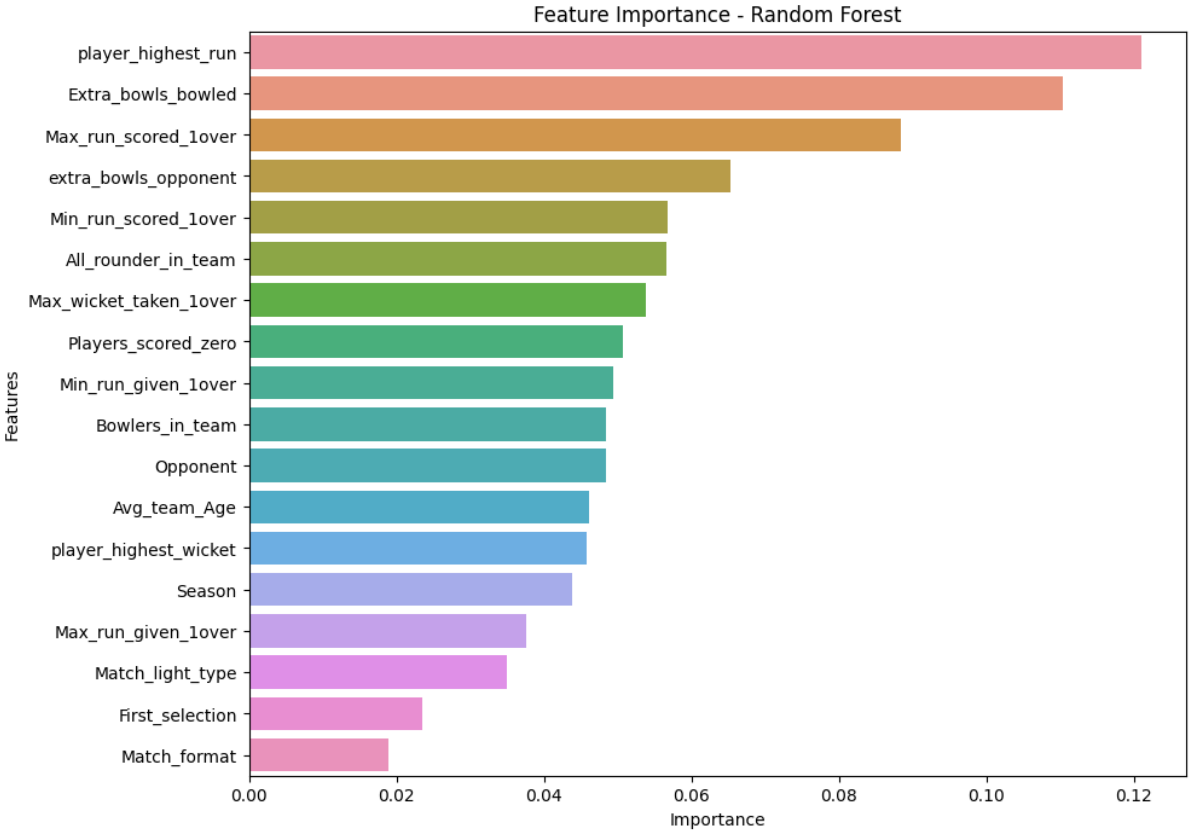
Feature Importance:

The feature importance analysis reveals critical insights into the factors that most influence match outcomes. By focusing on these key features, the cricket team can develop targeted strategies to enhance their performance, increase their chances of winning, and achieve their overall objective of maximizing victories and overcoming losses.

Rank Features by Importance:

- Features are ranked from most to least important based on their scores. This ranking helps to understand which variables the model relies on the most.

Fig 41 feature importance for random forest:



The feature importance plot from a Random Forest model highlights the relative importance of various features in predicting the target variable. Here are some business-related recommendations or insights based on the observed feature importances.

Business Insights/Recommendations:

Focus on Key Players: Features such as player_highest_run, player_highest_wicket, and Max_run_scored_1over are highly important. This suggests that the individual performance of key players has a significant impact on the outcome.

Recommendation: Invest in identifying and nurturing key players. Provide additional training and resources to high-performing players to maximize their impact.

Improve Bowling Performance: Features like Max_wicket_taken_1over, Extra_bowls_bowled, and extra_bowls_opponent also have high importance. Effective bowling strategies are crucial.

Recommendation: Focus on improving bowling techniques and strategies. Analyze and minimize the number of extra balls bowled, which can be a key factor in controlling the game.

Optimize Team Composition: Features such as Bowlers_in_team and All_rounder_in_team have noticeable importance.

Recommendation: Balance the team composition with a mix of specialized bowlers and all-rounders. This can provide both offensive and defensive capabilities, enhancing the team's overall performance.

Consider Match Conditions: Match_format, Match_light_type, and Season are also relevant features.

Recommendation: Tailor strategies based on the match format and conditions. For instance, different approaches may be required for day vs. night matches, or T20 vs. ODI formats. Seasonal conditions can affect player performance and game dynamics.

Strategize Based on Opponents: The feature Opponent has significant importance, indicating that the performance can vary depending on the opposing team.

Recommendation: Conduct in-depth analysis of past performances against specific opponents. Develop customized strategies to exploit their weaknesses and counter their strengths.

Leverage Experience and Selection: Features like First_selection and Avg_team_Age indicate the importance of experience and initial team selection.

Recommendation: Consider the experience level of players when making team selections. Experienced players might handle high-pressure situations better, contributing to better performance.

Minimize Poor Performances: Players_scored_zero is an important feature, suggesting that minimizing the number of players who score zero is critical.

Recommendation: Work on enhancing the consistency of all players. Provide targeted support to players who are prone to low scores to improve their reliability.

By focusing on these insights and recommendations, businesses (such as cricket teams or franchises) can develop more effective strategies, improve team performance, and potentially achieve better outcomes in their matches.

END OF BUSINESS REPORT